GOVERNMENT COLLEGE OF ENGINEERING

SALEM -636011

REGISTER NO	:	
SUBJECT	:	
DEPARTMENT	:	

Certified that this is the Bonafide record work done by Thiru/Selvi …… of ……. Semester of ECE Branch during the year 2025-26.

Date: ……. Staff-in-Charge

Submitted for the University Practical Examination held on ……... at the Government College of Engineering, Salem-11.

Internal Examiner External Examiner

INDEX

S.NO	DATE	Title	Page No	Marks	Signature
1.a		Design and Simulation of Combinational Circuits using Verilog HDL			
1.b		Design and Simulation of Serial Binary Adder and Carry Look-Ahead Adder using Verilog HDL			
1.c		Design and Simulation of Code Converters Using Verilog HDL			
2		Design and Simulate Sequential Circuits – 4-bit Binary Counter			
3		Design of Traffic Light Controller using Verilog HDL			
4		Design of Pipelined Parallel Adder to Add Eight 12-bit 2's-Complement Numbers			
5		Design 8-Bit Signed Multiplication Algorithm (2's- Complement)			
6		Study of FPGA			
7		Design and implementation of alu/mac unit in FPGA			
8		Implementation of flip-flops in FPGA			

INDEX

S.NO	DATE	Title	Page No	Marks	Signature
9		Embedded programs for i/o interfacing using pic controller			
10.a		Design a Stepper motor controller using PIC controller			
10.b		Design a DC motor controller using PIC controller			
11		Generate 3-phase PWM signals and demonstrate the utility of PWM with high bright LED lights using RL78			
12		Measure room temperature and display the same in a LCD with keyboard interaction using RL 78			
13		Design an Embedded system to measure the unknown signal frequency using Timer/Counter of RL78			
14		Demonstrate the usage of Watchdog timers and voltage detection facilities of RL78 in an application			
15		Interface ADC with embedded system trainer kit			
16		Interface 3-axis motion & vibration sensor with STM32 Nucleo board			

EX NO: 01 A. Design and Simulate Combinational circuits using Verilog HDL

DATE:

AIM:

To design and simulate the combinational circuits using Verilog HDL

a. 8:1 Multiplexer and 1:8 Demultiplexer.
b. Encoder 8:3 and Decoder 3:8

SOFTWARE USED:

Modelsim PE 5.5e.

THEORY:

MULTIPLEXER:

A Multiplexer functions as a multiple input, single output switch. It is also known as a data selector that selects between several analog or digital signals and forwards the selected input to a single output line.

A Multiplexer of 2^n input has n select lines which are used to select which input lines to send to the output.

DEMULTIPLEXER:

A Demultiplexer is a device taking a single input and selecting signals of the output of the compatible mux, which is connected to the single input, and a shared select lines. A demultiplexer is of n inputs and 2^n output.

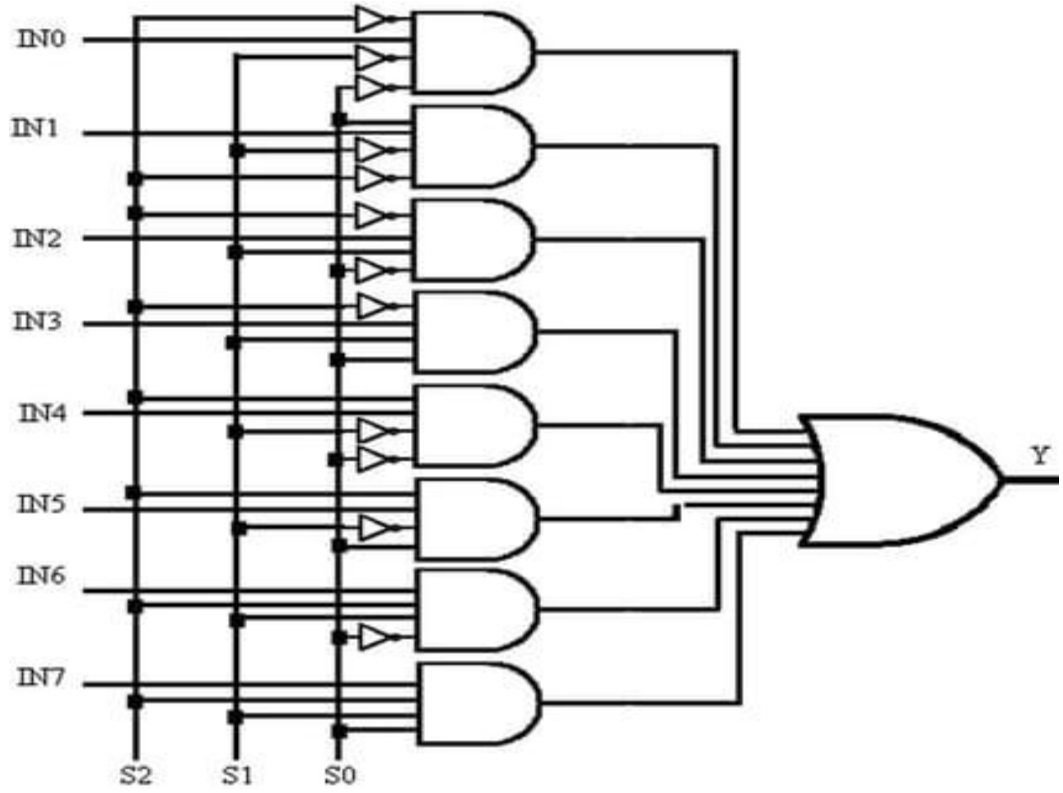
ENCODER:

An Encoder is a combinational circuit. It has 2^n input lines and n-output lines. It takes up these 2^n input data and encodes them into n-bit data. In other words, it produces the binary code equivalent of the input line, which is active high.

DECODER:

Decoder is a combinational circuit, it has n-input lines and 2^n output lines. It takes up these n-input data and decodes them into 2^n bit data. Based on the input, only one output line will be at logic high.

MULTIPLEXER CIRCUIT DIAGRAM:



TRUTH TABLE:

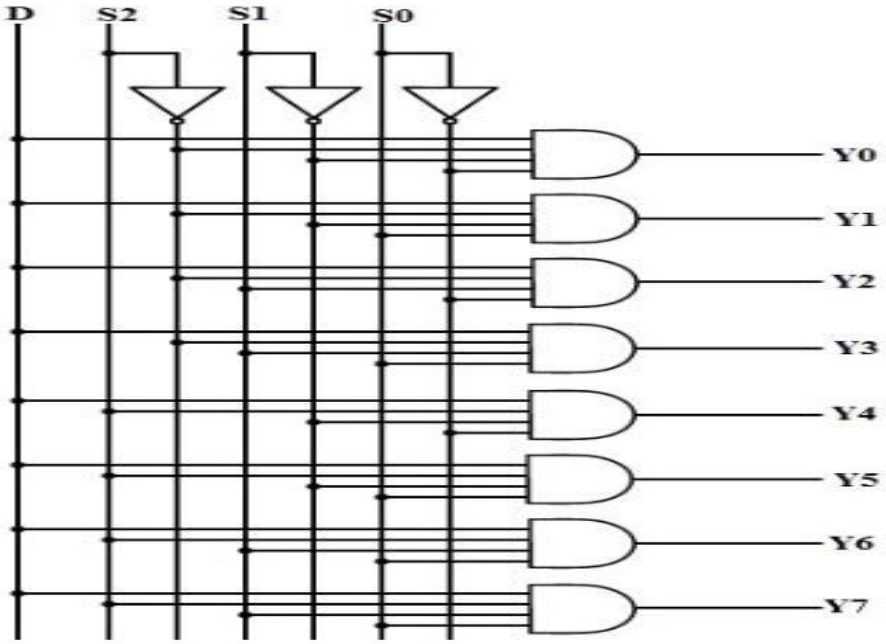
S2	S1	S0	OUT
0	0	0	D0
0	0	1	D1
0	1	0	D2
0	1	1	D3
1	0	0	D4
1	0	1	D5
1	1	0	D6
1	1	1	D7

VERILOG CODE FOR MULTIPLEXER:

BEHAVIORIAL MODELLING:

```
module mux (d0,d1,d2,d3,d4,d5,d6,d7,s0,s1,s2,out);
input d0,d1,d2,d3,d4,d5,d6,d7,s0,s1,s2;
output out;
wire t1,t2,t3,t4,t5,t6,t7,t8,t9,t10,t11;
not(t1,s0) ;
not(t2,s1);
not(t3,s2);
and(t1,s0);and(t2,s1);and(t3,s2);
and(t4,t3,t2,d0);
and(t5,t3,t2,s0,d1);
and(t6,t3,s1,t1,d2);
and(t7,t3,s0,s1,d3);
and(t8,d4,t1,t2,s2);
and(t9,d5,s0,t2,s2);
and(t10,d6,t1,s1,s2);
and(t11,d7,s0,s1,s2);
or(out,t4,t5,t6,t7,t8,t9,t10,t11);
endmodule
```

DEMULTIPLEXER CIRCUIT DIAGRAM:



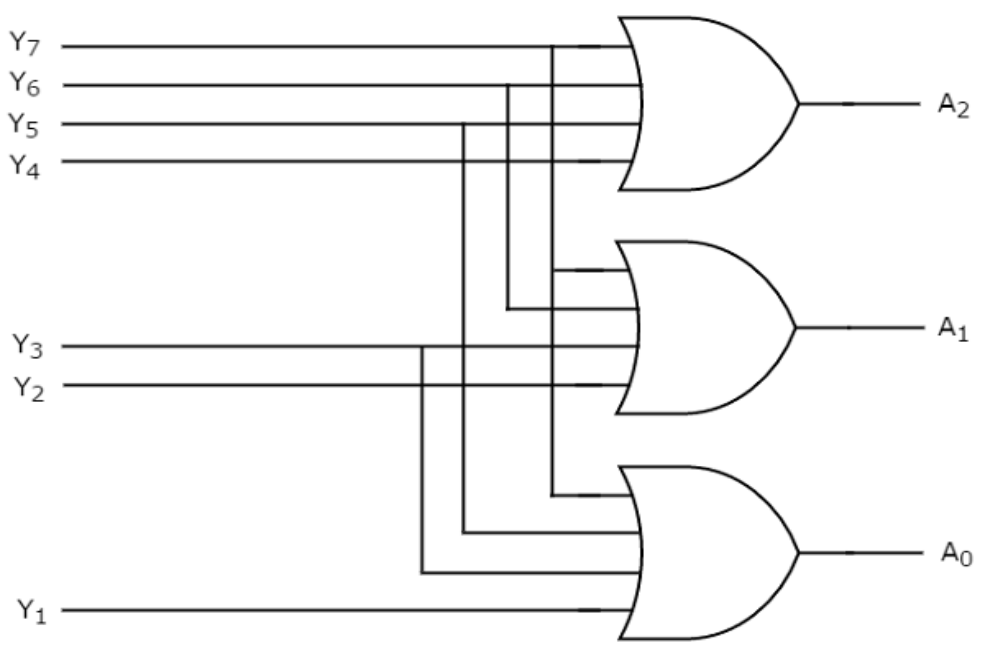
TRUTH TABLE:

D	**INPUT**			**OUTPUT**							
D	**S2**	**S1**	**S0**	**Y7**	**Y6**	**Y5**	**Y4**	**Y3**	**Y2**	**Y1**	**Y0**
D	0	0	0	0	0	0	0	0	0	0	D
D	0	0	1	0	0	0	0	0	0	D	0
D	0	1	0	0	0	0	0	0	D	0	0
D	0	1	1	0	0	0	0	D	0	0	0
D	1	0	0	0	0	0	D	0	0	0	0
D	1	0	1	0	0	D	0	0	0	0	0
D	1	1	0	0	D	0	0	0	0	0	0
D	1	1	1	D	0	0	0	0	0	0	0

VERILOG CODE FOR DEMULTIPLEXER:

```
module demultiplexer( d,s0,s1,s2,d0,d1,d2,d3,d4,d5,d6,d7);
input d,s0,s1,s2;
output d0,d1,d2,d3,d4,d5,d6,d7;
wire t1,t2,t3;
not(t1,s0);
not(t2,s1);
not(t3,s2);
and(d0,d1,t1,t2,t3);
and(d1,d,t2,t3,s0);
and(d2,d,t1,s1,t3);
and(d3,d,t0,s1,t3);
and(d4,d,s2,t2,s0);
and(d5,d,s2,t2,s0);
and(d6,d,t1,s1,s2);
and(d7,d,s0,s1,s2);
endmodule
```

ENCODER CIRCUIT DIAGRAM:



TRUTH TABLE:

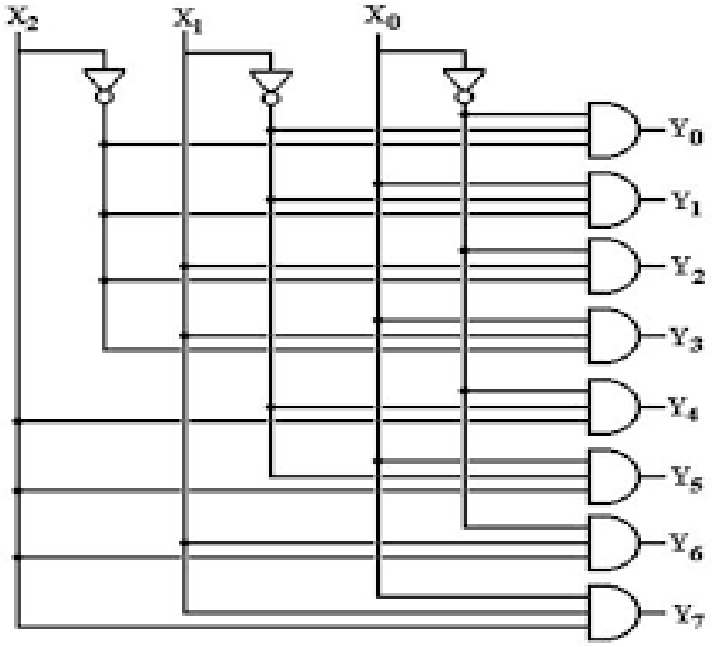
INPUT								OUTPUT		
Y7	**Y6**	**Y5**	**Y4**	**Y3**	**Y2**	**Y1**	**Y0**	**A2**	**A1**	**A0**
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

VERILOG CODE FOR ENCODER:

BEHAVIORIAL MODELLING:

```
module encoder8to3 (din,a,b,c);
input [0:7]din;
output a,b,c;
reg a,b,c;
always@(din)
begin
case(din)
8'b10000000:begin a=1'b0;b=1'b0,c=1'b0;end
8'b01000000:begin a=1'b0;b=1'b0;c=1'b1;end
8'b00100000:begin a=1'b0;b=1'b1;c=1'b0;end
8'b00010000:begin a=1'b0;b=1'b1;c=1'b1;end
8'b10001000:begin a=1'b1;b=1'b0,c=1'b0;end
8'b10000100:begin a=1'b1;b=1'b0,c=1'b1;end
8'b10000010:begin a=1'b1;b=1'b1,c=1'b0;end
8'b10000001:begin a=1'b1;b=1'b1,c=1'b1;end
default :begin a=1'bz;b=1'bz;c= 1'b1;end
endcase
end
endmodule
```

DECODER CIRCUIT DIAGRAM:



TRUTH TABLE:

INPUT			OUTPUT							
X2	X1	X0	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

VERILOG CODE FOR DECODER:

```
module decoder3to8( in, out, en);
input [2:0] in;
input en;
output [7:0] out;
  reg [7:0] out;

 always @(in or en)
          begin

      if (en)
        begin
          out=8'd0;
          case (in)
              3'b000: out[0]=1'b1;
              3'b001: out[1]=1'b1;
               3'b010: out[2]=1'b1;
              3'b011: out[3]=1'b1;
              3'b100: out[4]=1'b1;
              3'b101: out[5]=1'b1;
              3'b110: out[6]=1'b1;
              3'b111: out[7]=1'b1;
              default: out=8'd0;
          endcase
      end
else
out=8'd0;
end
endmodule
```

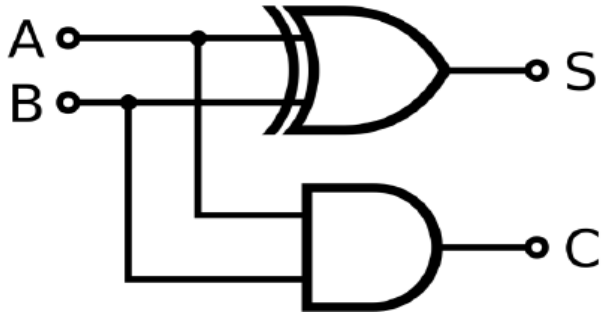
OUTPUT:

RESULT:

Thus, the Verilog HDL program for Multiplexer, Demultiplexer, encoder and Decoder was designed, simulated and corresponding waveforms are generated and verified.

DIAGRAM:

Half Adder:



Truth table:

Half Adder

Input1	Input2	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

EX.NO: 01 B Design and Simulate Serial Binary Adder And Look Ahead Adder

Date:

AIM:

To design And Simulate Serial Binary Adder and Look Ahead Adder using Verilog HDL.

SOFTWARE USED:

Xilinx ISE 9.2i software

THEORY:

HALF ADDER:

The half adder consists of two input variables designated as Augends and Addend bits. Output variables produce the Sum and Carry. The 'carry' output is 1 only when both inputs are 1 and, sum' is 1 if any one input is 1. The Boolean expression is given by,

sum = x ^ y carry = x & y

FULL ADDER:

A Full adder is a combinational circuit that focuses the arithmetic sum of three bits. It consists of 3 inputs and 2 outputs. The third input is the carry from the previous Lower Significant Position. The two outputs are designated as Sum (S) and Carry (C). The binary variable S gives the value of the LSB of the Sum. The output S=1 only if odd number of 1's are present in the input and the output C=1 if two or three inputs are 1.

sum = x ^ y ^ z

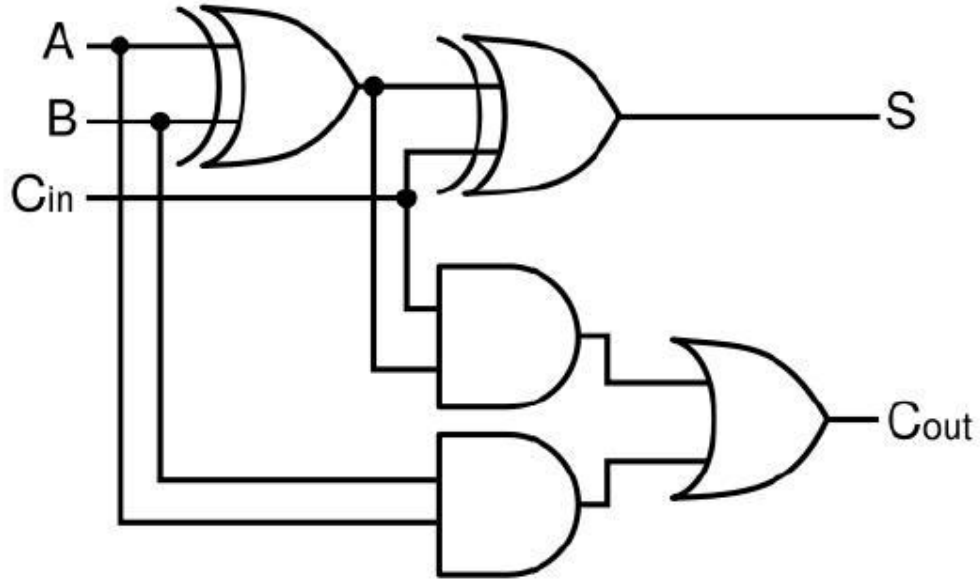
carry= (x & y) | (y & z) | (x & z)

Serial Binary Adder:

In the ripple carry adder, the output is known after the carry generated by the previous stage is produced. Thus, the sum of the most significant bit is only available after the carry signal has rippled through the adder from the least significant stage to the most significant stage. As a result, the final sum and carry bits will be valid after a considerable delay.

DIAGRAM:

Full Adder:



Truth Table:

a	b	c	carry	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

CARRY LOOK AHEAD ADDER:

The carry look ahead adder (CLA) solves the carry delay problem by calculating the carry signals in advance, based on the input signals. It is based on the fact that a carry signal will be generated in two cases: (1) when both bits a_i and b_i are 1, or (2) when one of the two bits is 1 and the carry-in is 1.

$$\begin{aligned} c_{i+1} &= G_i + P_i.c_i \\ s_i &= P_i \oplus c_i \end{aligned}$$

$$\begin{aligned} G_i &= a_i.b_i \\ P_i &= a_i \oplus b_i \end{aligned}$$

G_i and P_i are called the carry generate and carry propagate terms, respectively. The generate and propagate terms only depend on the input bits and thus will be valid after one and two gate delays.

$$\begin{aligned} c_1 &= G_0 + P_0.c_0 \\ c_2 &= G_1 + P_1.G_0 + P_1.P_0.c_0 \\ c_3 &= G_2 + P_2.G_1 + P_2.P_1.G_0 + P_2.P_1.P_0.c_0 \\ c_4 &= G_3 + P_3.G_2 + P_3.P_2.G_1 + P_3.P_2.P_1.G_0 + P_3.P_2.P_1.P_0.c_0 \end{aligned}$$

DIAGRAM:

SERIAL BINARY ADDER:

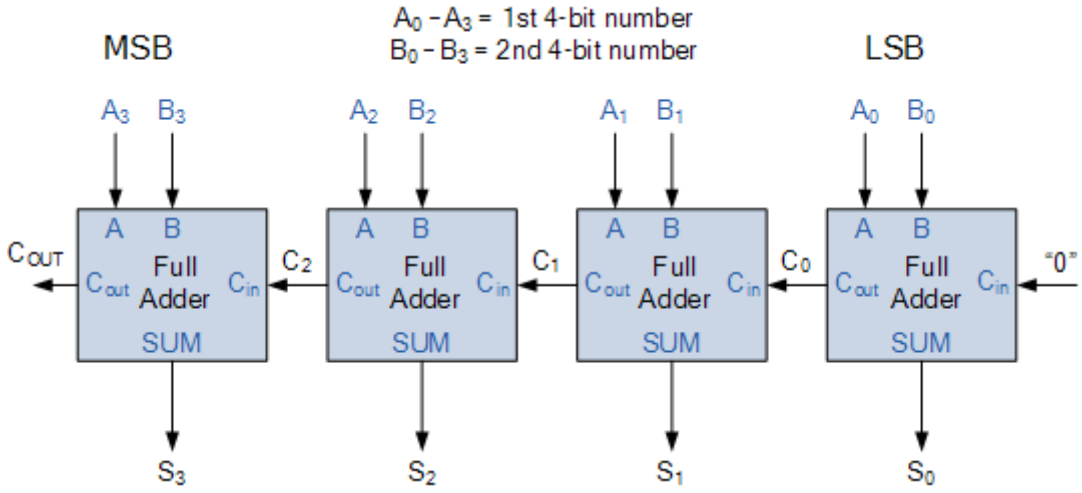
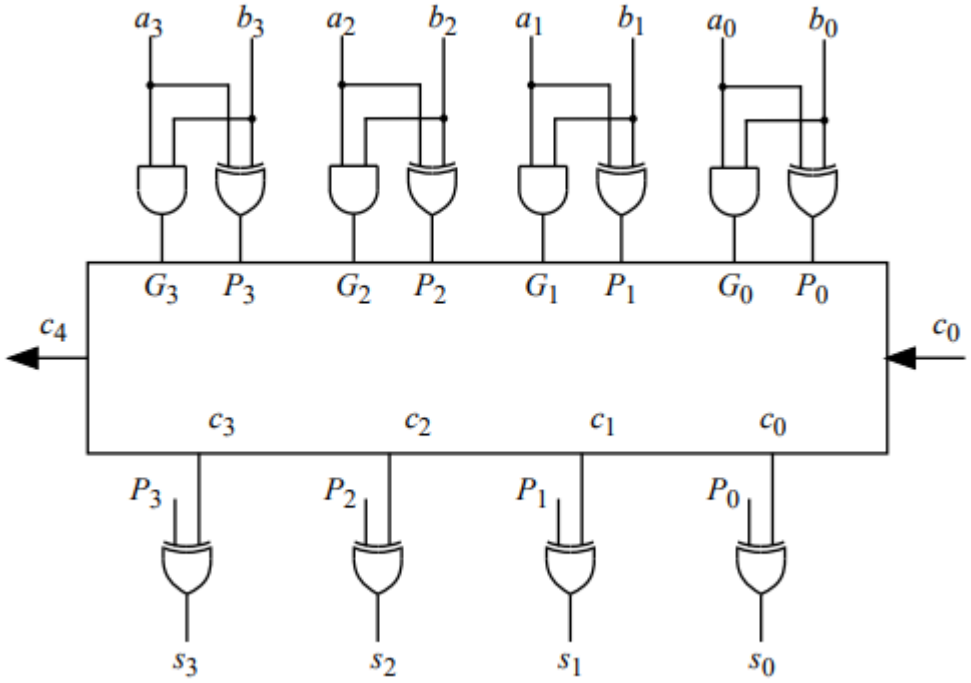


DIAGRAM:

CARRY LOOK AHEAD HEADER:



PROGRAM:

HALF ADDER

```
module ha(a,b,s,c);
input a,b;
output s,c;
assign s=a^b;
assign c=a&b;
endmodule
```

FULL ADDER

```
module fa(a,b,c,s,d);
input a,b,c;
output s,d;
assign s=a^b^c;
assign d=(a&b)|(b&c)|(c&a);
endmodule
```

SERIAL BINARY ADDER

```
module SBA(a,b,sum,carry,Cin);
output [3:0]sum;
output carry;
input [3:0]a,b;
wire [2:0]c;
fa u0(a[0],b[0],Cin,sum[0],c[0]);
fa u1(a[1],b[1],s[0],sum[1],c[1]);
fa u2(a[2],b[2],s[1],sum[2],c[2]);
fa u3(a[3],b[3],s[2],sum[3],carry);
endmodule
```

CARRY LOOK AHEAD ADDER:

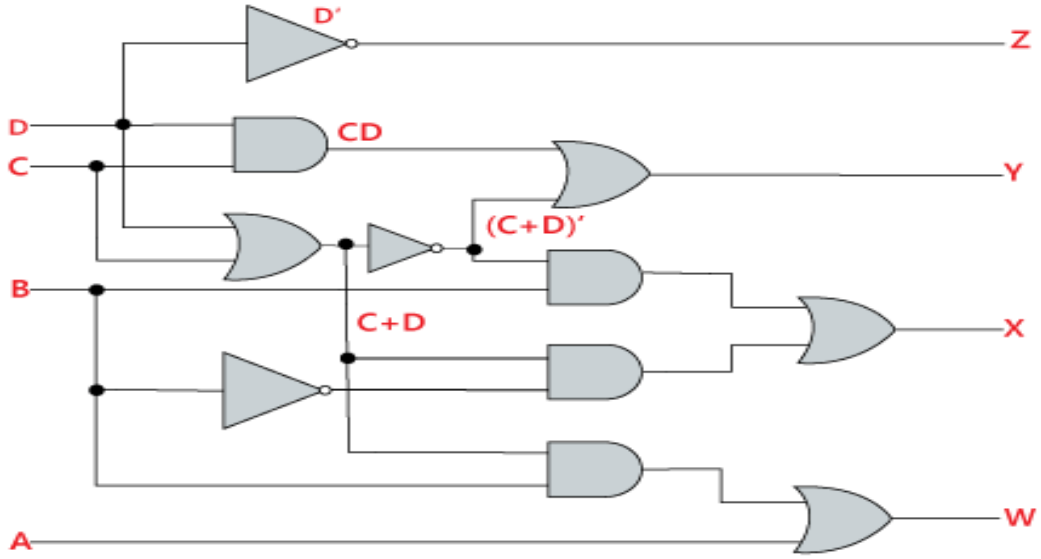
```
module cla_adder(input [3:0] A,B,input cin,output [3:0] S,output cout);
wire [3:0] P,G;
wire [4:0] C;
assign P = A ^ B;
assign G = A & B;
cla_block gen_c(P,G,cin,C);
assign S = P ^ C[3:0];
assign cout = C[4];
endmodule
module cla_block(input [3:0] P,G,input cin,output [4:0] C);
assign C[0] = cin;
assign C[1] = G[0] | (P[0] & cin);
assign C[2] = G[1] | (P[1] & G[0]) | (P[1] & P[0] & cin);
assign C[3] = G[2] | (P[2] & G[1]) | (P[2] & P[1] & G[0]) | (P[2] & P[1] & P[0] & cin);
assign C[4] = G[3] | (P[3] & G[2]) | (P[3] & P[2] & G[1]) | (P[3] & P[2] & P[1] & G[0]) | (P[3] &
P[2] & P[1] & P[0] & cin);
endmodule
```

OUTPUT:

RESULT:

Thus, the Verilog HDL program for serial binary adder and look ahead adder was designed, simulated and corresponding waveforms are generated and verified.

DIAGRAM:



TRUTH TABLE:

Decimal Digit	Input BCD				Output Excess-3			
	A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

EX NO: 1 C **Design and Simulate BCD To Excess 3 Code Converter.**

DATE:

AIM:

To design and simulate BCD to Excess 3 code converter in Verilog HDL.

SOFTWARE USED:

Xilinx ISE 9.2i software.

THEORY:

BCD COUNTER:

BCD counter is one of the 4-bit binary counters, which counts from 0 to a pre-determined count with an applied clock signal. When the count reaches the predetermined count value, it resets all the flip-flops and starts to count again from 0. This type of counter is designed by using 4 JK flip flops and counts from 0 to 9, and the result is represented in digital form. After reaching the count of 9 (1001), it resets and starts again.

EXCESS 3 CODE:

The excess-3 code (or XS3) is a non-weighted code used to express code used to express decimal numbers. It is a self-complementary binary coded decimal (BCD) code and numerical system which has biased representation. It is particularly significant for arithmetic operations as it overcomes shortcoming encountered while using 8421 BCD code to add two decimal digits whose sum exceeds 9. Excess-3 arithmetic uses different algorithm than normal non-biased BCD or binary positional number system.

OUTPUT:

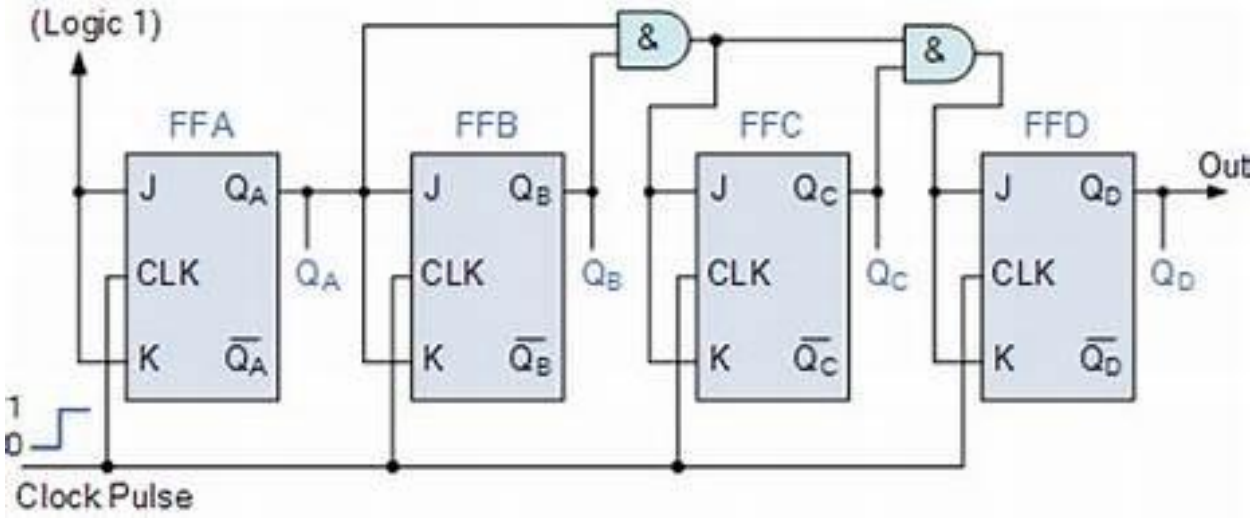
PROGRAM:

```
module bcdtoexcess3(input D0, D1, D2, D3, output, E0, E1, E2, E3);
assign E3=D3|D2&D0|D2&D1;
assign E2=(D2&(~D1) & (~D0)) | ((~D2) &D0) | ((~D2) & D1);
assign E1=((~D1) & (~D0)) | (D1&D0);
assign E0=~D0;
endmodule
```

RESULT:

Thus, the BCD to Excess 3 code converter were designed using Verilog HDL code and it was simulated and verified.

DIAGRAM:


(Logic 1)
&
&
FFA
FFB
FFC
FFD
J
CLK
K
Q_A
$\overline{Q_A}$
Q_B
$\overline{Q_B}$
Q_C
$\overline{Q_C}$
Q_D
$\overline{Q_D}$
Out
1
0
Clock Pulse

EX NO: 02 **Design and Simulate Sequential Circuits –**

4 bit Binary Counter

DATE:

AIM:

To design and simulate 4bit binary counter using JK flip flop in Verilog HDL.

SOFTWARE USED:

Xilinx ISE 9.2i software.

THEORY:

In the 4-bit binary Synchronous Counter, the external clock signal is connected to the clock input of every individual flip-flop within the counter so that all of the flip-flops are clocked together simultaneously (in parallel) at the same time. In other words, changes in the output occur in "synchronisation" with the clock signal. The result of this synchronisation is that all the individual output bits changing state at exactly the same time in response to the common clock signal with no ripple effect and therefore, no propagation delay.

PROGRAM:

JK Flip Flop

```
module jk(q,qb,clk,rst,j,k);
input clk,rst,j,k;
output q,qb;
reg q,qb;
always@(negedge clk)
begin
        if(rst==1'b1)
                begin
                        q=1'b0;
                        qb=1'b1;
                 end
         else if (clk==1'b0&rst==1'b0)
                 if (j==0&&k==0)
                        begin
                                q=q;
                                qb=!q;
                        end
                 if(j==0&&k==1)
                        begin
                                q=0;
                                qb=!q;
                        end
```

OUTPUT:

```
if(j==1&&k==0)
                        begin
                                q=1;
                                qb=!q;
                        end
                if(j==1&&k==1)
                        begin
                                q=qb;
                                qb=!q;
                        end
end
endmodule
```

4-BIT BINARY COUNTER:

```
module count(c,cb,clk,rst,J,K);
output [3:0]c,cb;
input clk,rst,J,K;
wire t1,t2;
assign t1=c[0]&c[1];
assign t2=t1&c[2];
JK a1(c[0],cb[0],clk,rst,J,K);
JK a2(c[1],cb[1],clk,rst,c[0],c[0]);
JK a3(c[2],cb[2],clk,rst,t1,t1);
JK a4(c[3],cb[3],clk,rst,t2,t2);
Endmodule
```

RESULT:

Thus the 4-bit synchronous binary counter using JK flipflop were designed using verilog HDL code and it was simulated and verified.

EX NO: 03 Design of Traffic Light Controller Using Verilog HDL

DATE:

AIM:

To design a traffic light controller using Verilog HDL.

SOFTWARE USED:

Xilinx ISE 9.2i software

THEORY:

Consider a controller for traffic at the intersection of four roads. Consider P1, P2, P3, and P4 as four roads and P5 as pedestrian. The road has following states.

- GREEN-11001
- YELLOW-01000
- RED-00100

The pedestrian

- GREEN-0000
- Red-1111

First the road p1 is green and all other roads p2, p3, p4 and P5 are red. After a some delay p1 is turn to yellow and then red the traffic signal on p2 is green. After a delay p2 is turn to yellow and then red the signal on p3 is green. Then p3 is changed to yellow and then red the signal on p4 is changed to green. After a delay p4 is turn to yellow and then red the pedestrian light P5 is green after a delay P5 is turn to green then routine will continue.

PROGRAM:

```
module tlc( clk,reset,p1,p2,p3,p4,p5);
input clk;
input reset;
output[4:0]p1;
output[4:0]p2;
output[4:0]p3;
output[4:0]p4;
output[4:0]p5;
reg [4:0]p1;
reg[4:0]p2;
reg[4:0]p3;
reg[4:0]p4;
reg[3:0]p5;
reg[5:0]Sig;
always@(posedgeclk)
```

```
begin
        if(reset==1'b0)
begin
        p1<=5'b00100;
        p2<=5'b00100;
        p3<=5'b00100;
        p4<=5'b00100;
        p5<=4'b1111;
        Sig<=6'b000000;
end
        else begin
        Sig <= Sig+1;
        case(Sig[5:0])
        6'b000000:begin
        p1<=5'b11001;//GREEN
        p2<=5'b00100;
        p3<=5'b00100;
        p4<=5'b00100;
        p5<=4'b1111;
end
        6'b000100:begin
        p1<=5'b01000;//YELLOW
        p2<=5'b00100;
        p3<=5'b00100;
        p4<=5'b00100;
        p5<=4'b1111;
end
6’b001000:begin
p1<=5’b00100;
p2<=5’b11001;//GREEN
p3<=5’b00100;
p4<=5’b00100;
p5<=4’b1111;
end
6'b001100:begin
        p1<=5'b00100;
        p2<=5'b01000;//YELLOW
        p3<=5'b00100;
        p4<=5'b00100;
        p5<=4'b1111;
end
        6'b01000:begin
        p1<=5'b00100;
        p2<=5'b00100;
        p3<=5'b11001;//GREEN
        p4<=5'b00100;
        p5<=4'b1111;
end
        6'b010100:begin
        p1<=5'b00100;
```

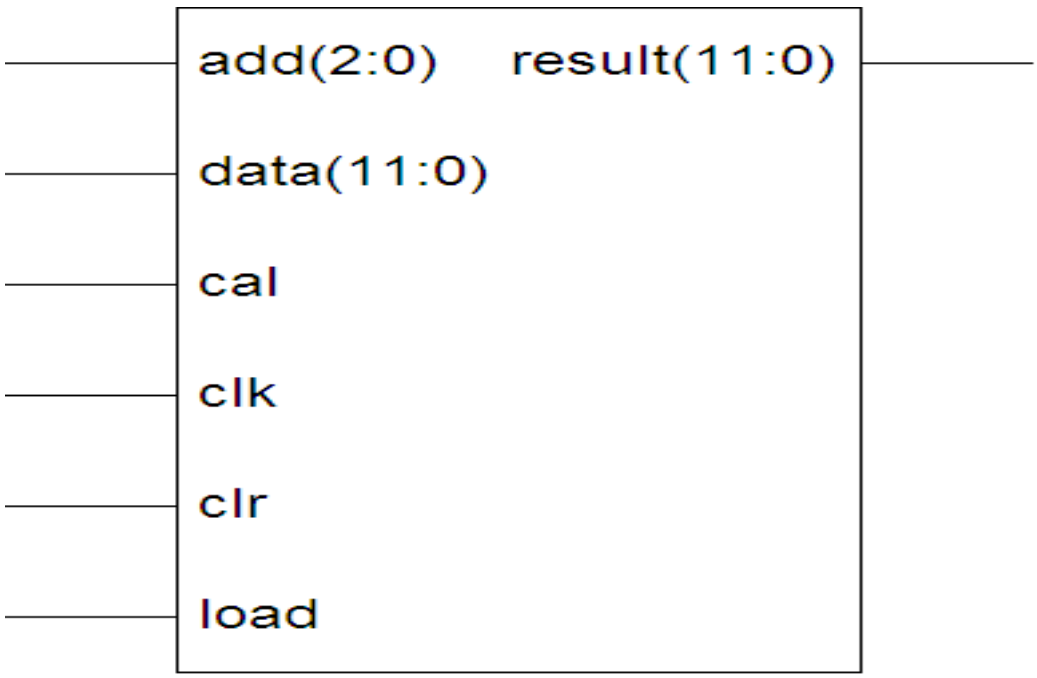
OUTPUT:

```
p2<=5'b00100;
          p3<=5'b01000;//YELLOW
          p4<=5'b00100;
          p5<=4'b1111;
end
6'b011000:begin
p1<=5'b00100;
p2<=5'b00100;
p3<=5'b00100;
p4<=5'b11001;//GREEN
p5<=4'b1111;
end
        6’b011100:begin
        p1<=5’b00100;
        p2<=5’b00100;
        p3<=5’b00100;
        p4<=5’b01000;  //YELLOW
        p5<=4’b1111;
end
6'b10000:begin
p1<=5'b00100;
p2<=5'b00100;
p3<=5'b00100;
p4<=5'b00100;
p5<=4'b0000;//PEDESTRAIN
end
6'b100100:Sig<=6'b000000;
default:begin
end
endcase
end
end
endmodule
```

RESULT:

Thus the traffic light controller was designed using verilog and simulated successfully.

PIN DIAGRAM:



EX NO: 04 Design Pipelined Parallel Adder to Add 8 Number of Size 12 bits in 2's Complement

DATE:

AIM:

To design and simulate pipelined parallel adder to add 8 number of size 12 bits in 2's Compliment.

SOFTWARE REQUIRED:

Xilinx ISE 9.2i software

THEORY:

Parallel adder is an adder which adds all the n-bits of m-numbers at a time. Addition is a fundamental operation for any digital system, digital signal processing or control system. A fast and accurate operation of a digital system is greatly influenced by the performance of the resident adders. Adders are also very important component in digital systems because of their extensive use in other basic digital operations such as subtraction, multiplication and division. Hence, improving performance of the digital adder would greatly advance the execution of binary operations inside a circuit compromised of such blocks. The performance of a digital circuit block is gauged by analyzing its power dissipation, layout area and its operating speed.

OUTPUT:

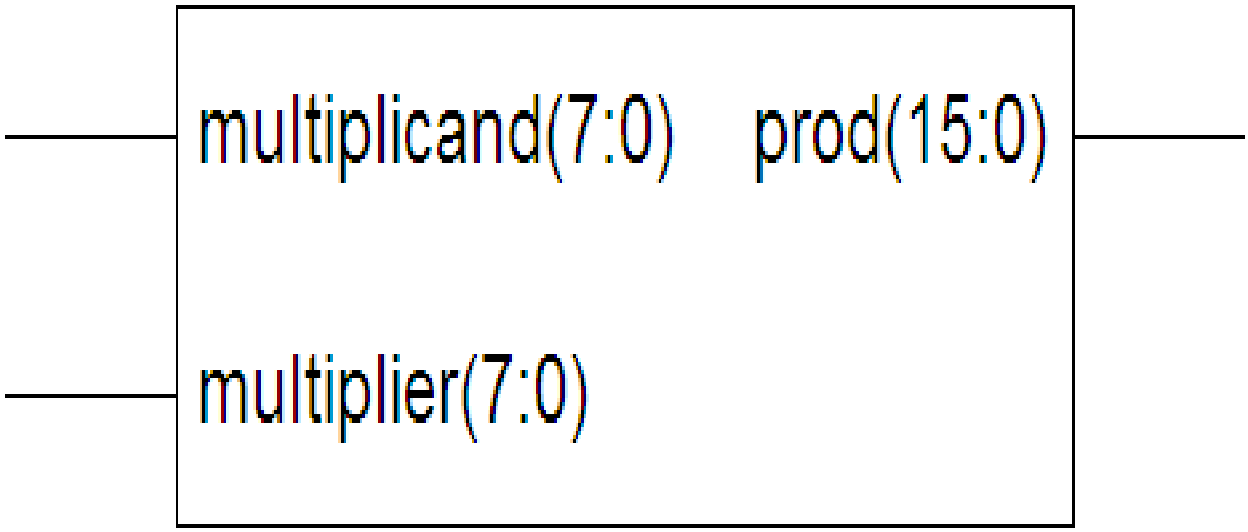
PROGRAM:

```
module parallel(clk,add,load,clr,data,cal,result);
inputclk,clr,cal,load;
input[2:0]add;
input[11:0]data;
outputreg[11:0]result;
reg[11:0]ram[7:0];
wire[11:0]tem;
always@(posedgeclk)
begin
if(~clr)
begin
ram[0]=12'b0;
ram[1]=12'b0;
ram[2]=12'b0;
ram[3]=12'b0;
ram[4]=12'b0;
ram[5]=12'b0;
ram[6]=12'b0;
ram[7]=12'b0;
end
else if(~load)
ram[add]=data;
end
assign tem=ram[0]+ram[1]+ram[2]+ram[3]+ram[4]+ram[5]+ram[6]+ram[7];
always@(posedgeclk)
begin
if(~load)
result=data;
else if(~cal)
result=tem;
else
result=ram[add];
end
end module
```

RESULT:

Thus the Pipelined Parallel Adder was designed to add 8 numbers of 12 bits each by using Verilog, and then output was verified.

DIAGRAM:



EX NO: 5 Design 8-Bit Signed Multiplication Algorithm

DATE:

AIM:

To design and simulate Verilog files for Multiplying two 8 bit numbers.

SOFTWARE REQUIRED:

Xilinx ISE 9.2i software

THEORY:

- Assign the clock input to clock and set values to multiplier and multiplicand.
- Check the multiplier and multiplicand are positive, then do the successive addition for the product.
- Otherwise multiplier is positive and multiplicand is negative, and then do the successive addition but in successive addition, MSB is replaced by '1' instead of '0'.
- If multiplier is negative and multiplicand is positive, then take 2's compliment of the multiplicand and do the successive addition for that multiplicand.
- If both multiplier & multiplicand are negative, then take 2's compliment of the multiplicand and do the successive addition, but MSB is replaced by '1' instead of '0'.

OUTPUT:

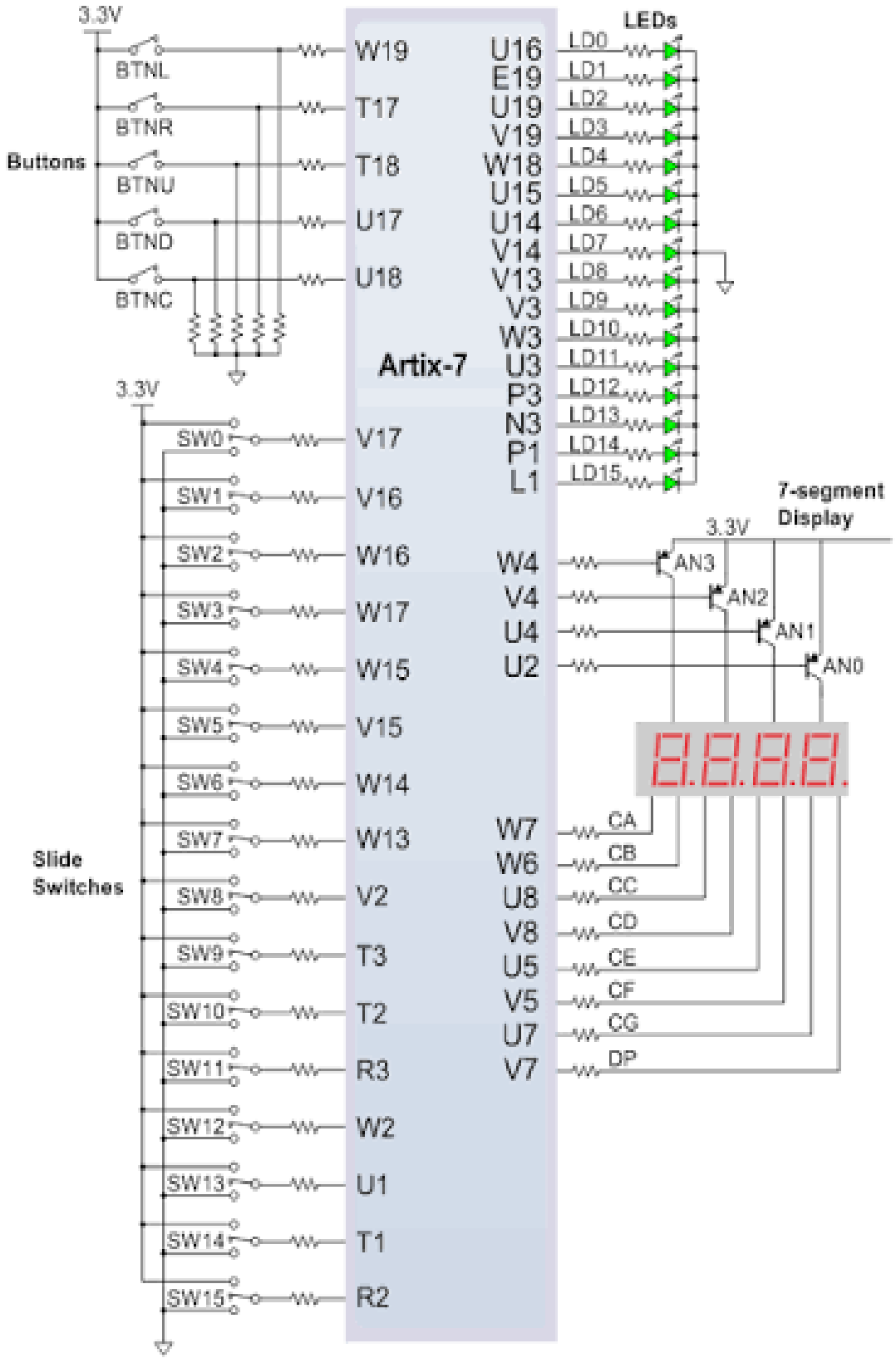
PROGRAM:

```
Module multi_signed(prod,multiplicand,multiplier);
    output [15:0] prod;
    input [7:0] multiplicand;
    input [7:0] multiplier;
    wire [15:0] shift1,shift2,shift3,shift4,shift5,shift6,shift7,shift8;
    wire[15:0] add1,add2,add3,add4,add5,add6,add7,add8;
    wire [15:0] prod_s;
    assign shift1 ={8'b0, multiplicand};
    assign shift2 ={7'b0, multiplicand,1'b0};
    assign shift3 ={6'b0, multiplicand,2'b0};
    assign shift4 ={5'b0, multiplicand,3'b0};
    assign shift5 ={4'b0, multiplicand,4'b0};
    assign shift6 ={3'b0, multiplicand,5'b0};
    assign shift7 ={2'b0, multiplicand,6'b0};
    assign shift8 ={1'b0, multiplicand,7'b0};
    assign add1 =(multiplier [0] == 1'b1)? shift1 : 16'b0;
    assign add2 =(multiplier [1] == 1'b1)? shift2 : 16'b0;
    assign add3 =(multiplier [2] == 1'b1)? shift3 : 16'b0;
    assign add4 =(multiplier [3] == 1'b1)? shift4 : 16'b0;
    assign add5 =(multiplier [4] == 1'b1)? shift5 : 16'b0;
    assign add6 =(multiplier [5] == 1'b1)? shift6 : 16'b0;
    assign add7 =(multiplier [6] == 1'b1)? shift7 : 16'b0;
    assign add8 =(multiplier [7] == 1'b1)? shift8 : 16'b0;
    assign prod_s = add1+add2+add3+add4+add5+ add6+add7+add8;
    assign prod=prod_s;
    endmodule
```

RESULT:

Thus the design for Verilog files for multiplying two 8 bit numbers was simulated and verified.

SCHEMATIC DIAGRAM:



EX NO: 6 STUDY OF FPGA BOARD.

DATE:

AIM:

To study the Basys3 Artix-7 FPGA Board trainer kit and testing the on-board LEDs and switches using VIVADO2023 software tool.

REQUIREMENTS:

S. NO	Item Name	Quantity
1.	PC	01
2.	VIVADO 2023 software	01
3.	Basys3 Artix-7 FPGA board	01
4.	Power Adapter	01
5.	Power Cable	01

THEORY:

BASYS3 ARTIX-7 FPGA BOARD:

The Basys3 is a digital circuit development board based on the Xilinx Artix-7 FPGA. It provides onboard switches, push buttons, LEDs, and a seven-segment display for testing and implementing digital designs.

DIP SWITCHES AND KEY SWITCHES:

The DIP switches and push buttons are used to provide digital input signals to the FPGA. Their states can be read by the FPGA and used as inputs for the designed logic circuit.

ON-BOARD LEDs:

The LEDs are connected to FPGA pins and are used to display the output of the implemented digital design. By assigning FPGA pins to the switches and LEDs in Vivado 2023, the switch inputs can be used to control **the corresponding LED**

PIN DESCRIPTION:

PINOUT DETAILS FOR SWITCHES:

SWITCHES	PINOUTS
Switch(SW0)	V17
Switch(SW1)	V16
Switch(SW2)	W16
Switch(SW3)	W17
Switch(SW4)	W15
Switch(SW5)	V15
Switch(SW6)	W14
Switch(SW7)	W13
Switch(SW8)	W12
Switch(SW9)	T3
Switch(SW10)	T2
Switch(SW11)	R3
Switch(SW12)	W2
Switch(SW13)	U1
Switch(SW14)	T1
Switch(SW15)	R2

PINOUT DETAILS FOR LEDS:

LEDS	PINOUTS
Led(LD0)	U16
Led(LD1)	E19
Led(LD2)	U19
Led(LD3)	V19
Led(LD4)	W18
Led(LD5)	U15
Led(LD6)	U14
Led(LD7)	V14
Led(LD8)	V13
Led(LD9)	V3
Led(LD10)	W3
Led(LD11)	U3
Led(LD12)	P3
Led(LD13)	N3
Led(LD14)	P1
Led(LD15)	L1

PROGRAM:

Source code:

```
module led_blink (
   input wire clk,          // 100 MHz input clock
   output reg [15:0] led    // 16 LEDs output
);

   parameter CLK_FREQ = 100_000_000; // 100 MHz
   parameter BLINK_FREQ = 1;         // 1 Hz blink rate

   localparam integer COUNT_MAX = CLK_FREQ / (2 * BLINK_FREQ);

   reg [31:0] counter = 0;

   always @(posedge clk)
   begin
      if (counter >= COUNT_MAX - 1)
      begin
         counter <= 0;
         led <= ~led; // Toggle all 16 LEDs
      end
      else
      begin
         counter <= counter + 1;
      end
   end
endmodule
```

Constraint code:

```
##clock signal
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]

##LED connections

set_property PACKAGE_PIN U16 [get_ports {led[0]}]
set_property PACKAGE_PIN E19 [get_ports {led[1]}]
set_property PACKAGE_PIN U19 [get_ports {led[2]}]
set_property PACKAGE_PIN V19 [get_ports {led[3]}]
set_property PACKAGE_PIN W18 [get_ports {led[4]}]
set_property PACKAGE_PIN U15 [get_ports {led[5]}]
set_property PACKAGE_PIN V14 [get_ports {led[6]}]
set_property PACKAGE_PIN U14 [get_ports {led[7]}]
set_property PACKAGE_PIN V13 [get_ports {led[8]}]
set_property PACKAGE_PIN V3 [get_ports {led[9]}]
set_property PACKAGE_PIN W3 [get_ports {led[10]}]
set_property PACKAGE_PIN U3 [get_ports {led[11]}]
```

PINOUT DETAILS FOR PUSH BUTTONS:

PUSH BUTTONS	PINOUTS
Push Button(BTNU)	T18
Push Button(BTNL)	W19
Push Button(BTND)	U17
Push Button(BTNR)	T17
Push Button(BTNC)	U18

PINOUT DETAILS FOR SEVEN SEGMENTS:

SEVEN SEGMENT PINS	PINOUTS
Seven Segment(seg[0])	W7
Seven Segment(seg[1])	W6
Seven Segment(seg[2])	U8
Seven Segment(seg[3])	V8
Seven Segment(seg[4])	U5
Seven Segment(seg[5])	V5
Seven Segment(seg[6])	U7

DECIMAL POINTER PIN:

DECIMAL POINTER	PINOUTS
Decimal Pointer	V7

ANODE CONTROL PINS:

ANODE CONTROL	PINOUTS
an[0]	U2
an[1]	U4
an[2]	V4
an[3]	W4

```
set_property PACKAGE_PIN P3 [get_ports {led[12]}]
set_property PACKAGE_PIN N3 [get_ports {led[13]}]
set_property PACKAGE_PIN P1 [get_ports {led[14]}]
set_property PACKAGE_PIN L1 [get_ports {led[15]}]

## IOSTANDARD for all LEDs

set_property IOSTANDARD LVCMOS33 [get_ports {led[*]}]
```

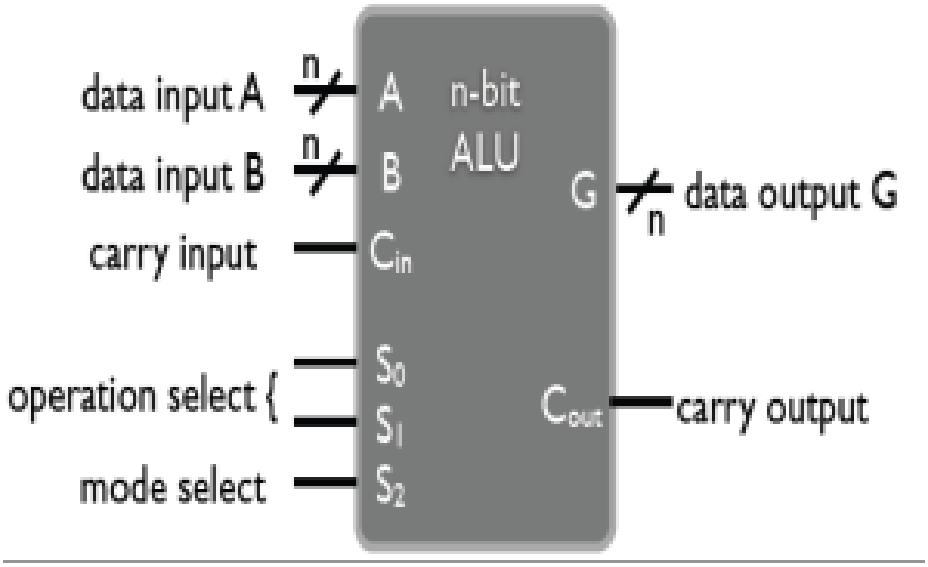
PROCEDURE:

1. Connect the Basys3 Artix-7 FPGA board to the PC and open Vivado 2023.
2. Create a new RTL project and select the appropriate Basys3 Artix-7 FPGA device as the target device.
3. Write the Verilog HDL program to take input from the onboard switches and control the LEDs.
4. Add the required XDC constraint file and assign the FPGA pins for the switches and LEDs.
5. Run Synthesis and Implementation, generate the Bitstream, and program the FPGA using Vivado Hardware Manager.
6. Operate the onboard switches and observe the corresponding LED outputs to verify that the LEDs function according to the designed logic.

RESULT:

Thus, the Basys3 Artix-7 FPGA Board trainer kit was studied and onboard LEDs and switches were tested using VIVADO2023 software tool.

BLOCK DIAGRAM:



EX NO: 7 **DESIGN AND IMPLEMENTATION OF ALU/MAC UNIT IN FPGA**

DATE:

AIM:

To design and implement ALU unit in Basys3 Artix-7 FPGA board trainer kit and verify the same in VIVADO2023 software tool.

REQUIREMENTS:

S. NO	Item Name	Quantity
1.	PC	01
2.	VIVADO2023 software tool	01
3.	Basys3 Artix-7 FPGA board	01
3.	Power Adapter	01
4.	Power Cable	01

THEORY:

ALU (ARITHMETIC LOGIC UNIT): An ALU is a digital circuit that performs arithmetic and logical operations on binary data. It is an important part of a processor and can perform operations such as addition, subtraction, AND, OR, and XOR.

ALU IMPLEMENTATION IN FPGA: The ALU can be designed using Verilog HDL and implemented on the Basys3 Artix-7 FPGA. The input operands and operation selection are given through the FPGA inputs, while the result is displayed using the onboard LEDs or other output devices.

DESIGN AND VERIFICATION: The ALU design is synthesized and implemented using Vivado 2023. The generated bitstream is programmed into the FPGA, and different input combinations are tested to verify the correctness of the arithmetic and logical operations.

PIN DESCRIPTION:

PINOUT DETAILS FOR SWITCHES:

SWITCHES	PINOUTS
Switch(SW0)	V17
Switch(SW1)	V16
Switch(SW2)	W16
Switch(SW3)	W17
Switch(SW4)	W15
Switch(SW5)	V15
Switch(SW6)	W14
Switch(SW7)	W13
Switch(SW8)	W12
Switch(SW9)	T3
Switch(SW10)	T2
Switch(SW11)	R3
Switch(SW12)	W2
Switch(SW13)	U1
Switch(SW14)	T1
Switch(SW15)	R2

PINOUT DETAILS FOR LEDS:

LEDS	PINOUTS
Led(LD0)	U16
Led(LD1)	E19
Led(LD2)	U19
Led(LD3)	V19
Led(LD4)	W18
Led(LD5)	U15
Led(LD6)	U14
Led(LD7)	V14
Led(LD8)	V13
Led(LD9)	V3
Led(LD10)	W3
Led(LD11)	U3
Led(LD12)	P3
Led(LD13)	N3
Led(LD14)	P1
Led(LD15)	L1

PROGRAM:

Source code:

```
module alu (

   input clk,
   input sel_load,
   input [3:0] A, B,
   input [3:0] ALU_Sel_in,
   output [3:0] ALU_Out,
   output CarryOut
);

   reg [3:0] ALU_Result;
   reg [3:0] ALU_Sel;
   wire [4:0] tmp;

   assign tmp = {1'b0, A} + {1'b0, B};
   assign CarryOut = tmp[4];
   assign ALU_Out = ALU_Result;

   always @(posedge clk)
   begin
      if (sel_load)
      begin
         ALU_Sel <= ALU_Sel_in;
      end
   end

   always @(*)
   begin
      case (ALU_Sel)
         4'b0000: ALU_Result = A + B;
         4'b0001: ALU_Result = A - B;
         4'b0010: ALU_Result = A * B;
         4'b0011: ALU_Result = (B != 0) ? A / B : 4'b0;
         4'b0100: ALU_Result = A << 1;
         4'b0101: ALU_Result = A >> 1;
         4'b0110: ALU_Result = {A[2:0], A[3]};
```

PINOUT DETAILS FOR PUSH BUTTONS:

PUSH BUTTONS	PINOUTS
Push Button(BTNU)	T18
Push Button(BTNL)	W19
Push Button(BTND)	U17
Push Button(BTNR)	T17
Push Button(BTNC)	U18

PINOUT DETAILS FOR SEVEN SEGMENTS:

SEVEN SEGMENT PINS	PINOUTS
Seven Segment(seg[0])	W7
Seven Segment(seg[1])	W6
Seven Segment(seg[2])	U8
Seven Segment(seg[3])	V8
Seven Segment(seg[4])	U5
Seven Segment(seg[5])	V5
Seven Segment(seg[6])	U7

DECIMAL POINTER PIN:

DECIMAL POINTER	PINOUTS
Decimal Pointer	V7

ANODE CONTROL PINS:

ANODE CONTROL	PINOUTS
an[0]	U2
an[1]	U4
an[2]	V4
an[3]	W4

```
4'b0111: ALU_Result = {A[0], A[3:1]};
          4'b1000: ALU_Result = A & B;
          4'b1001: ALU_Result = A | B;
          4'b1010: ALU_Result = A ^ B;
          4'b1011: ALU_Result = ~(A | B);
          4'b1100: ALU_Result = ~(A & B);
          4'b1101: ALU_Result = ~(A ^ B);
          4'b1110: ALU_Result = (A > B) ? 4'd1 : 4'd0;
          4'b1111: ALU_Result = (A == B) ? 4'd1 : 4'd0;
          default: ALU_Result = 4'd0;
       endcase
   end
endmodule
```

Constraint code:

```
 ## CLOCK (100 MHz onboard oscillator)

set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]

## SELECTION LOAD BUTTON (BTNU for example)

set_property PACKAGE_PIN U18 [get_ports sel_load]
set_property IOSTANDARD LVCMOS33 [get_ports sel_load]

## INPUT A[3:0] — SW0 to SW3

set_property PACKAGE_PIN V17 [get_ports {A[0]}]
set_property PACKAGE_PIN V16 [get_ports {A[1]}]
set_property PACKAGE_PIN W16 [get_ports {A[2]}]
set_property PACKAGE_PIN W17 [get_ports {A[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {A[*]}]

## INPUT B[3:0] — SW4 to SW7

set_property PACKAGE_PIN W15 [get_ports {B[0]}]
set_property PACKAGE_PIN V15 [get_ports {B[1]}]
set_property PACKAGE_PIN W14 [get_ports {B[2]}]
set_property PACKAGE_PIN W13 [get_ports {B[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {B[*]}]
```

```
## INPUT ALU_Sel_in[3:0] — SW8 to SW11

set_property PACKAGE_PIN V2 [get_ports {ALU_Sel_in[0]}]
set_property PACKAGE_PIN T3 [get_ports {ALU_Sel_in[1]}]
set_property PACKAGE_PIN T2 [get_ports {ALU_Sel_in[2]}]
set_property PACKAGE_PIN R3 [get_ports {ALU_Sel_in[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {ALU_Sel_in[*]}]

## OUTPUT ALU_Out[3:0] — LD0 to LD3

set_property PACKAGE_PIN U16 [get_ports {ALU_Out[0]}]
set_property PACKAGE_PIN E19 [get_ports {ALU_Out[1]}]
set_property PACKAGE_PIN U19 [get_ports {ALU_Out[2]}]
set_property PACKAGE_PIN V19 [get_ports {ALU_Out[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {ALU_Out[*]}]

## OUTPUT CarryOut — LD4

set_property PACKAGE_PIN W18 [get_ports CarryOut]
set_property IOSTANDARD LVCMOS33 [get_ports CarryOut]
```

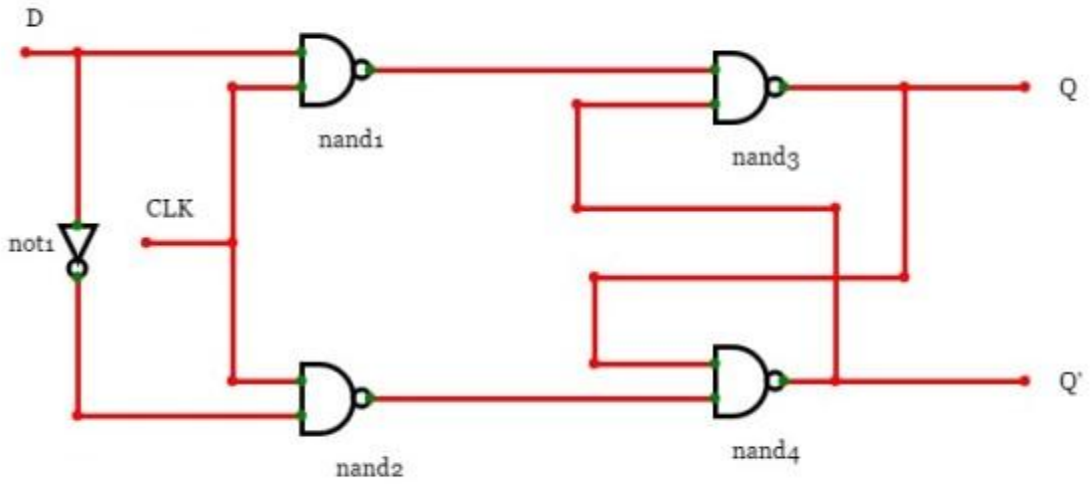
PROCEDURE:

1. Connect the Basys3 Artix-7 FPGA board to the PC and open Vivado 2023.
2. Create a new RTL project and select the appropriate Basys3 Artix-7 FPGA device as the target device.
3. Write the Verilog HDL program to design the ALU and define the required arithmetic and logical operations.
4. Add the XDC constraint file and assign the FPGA pins for the input operands, operation selection, and output signals.
5. Run Synthesis and Implementation, generate the Bitstream, and program the FPGA using Vivado Hardware Manager.
6. Apply different input combinations and operation selections, and observe the output to verify the correct functioning of the ALU.

Result:

Thus, the program for Arithmetic and Logic unit is simulated and implemented in the FPGA board.

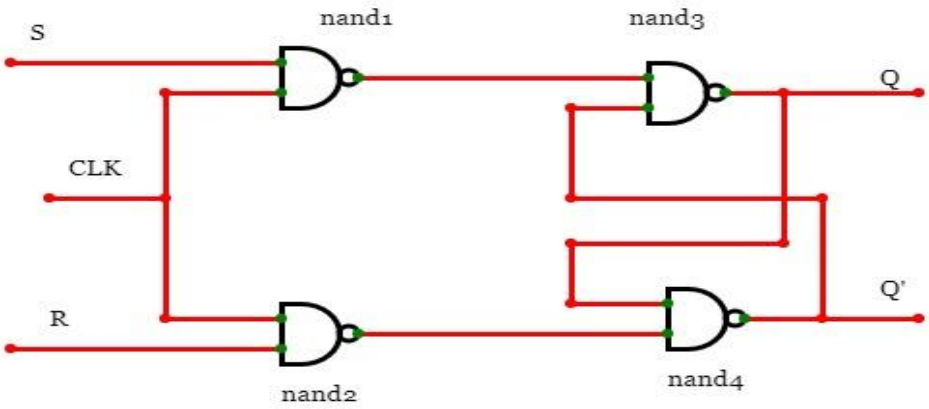
SR FLIP FLOP:



TRUTH TABLE:

S	R	Q	Q'
0	0	0	1
0	1	0	1
1	0	1	0
1	1	infinite	infinite

D FLIP FLOP:



EX NO: 8 IMPLEMENTATION OF FLIP-FLOPS IN FPGA

DATE:

AIM:

To simulate various flipflops using Verilog HDL and obtain waveforms.

THEORY:

FLIPFLOPS

Flip-flops are fundamental building blocks of sequential circuits. A flip flop can store one bit of data. Hence, it is known as a memory cell. Since they work on the application of a clock signal, they come under the category of synchronous circuits.

SR Flipflop

The SR flip-flop is one of the simplest basic flip-flops used for storing a single bit of information. It has two inputs, S (Set) and R (Reset), along with a clock input and two outputs, Q and $\overline{Q}$. When S = 1 and R = 0, the flip-flop is set, while S = 0 and R = 1 resets the flip-flop. When both inputs are 0, the previous state is retained. The condition S = R = 1 is an invalid condition in the basic SR flip-flop**.**

D Flipflop

A D flip-flop stands for data or delay flip-flop. The outputs of this flip-flop are equal to the inputs.

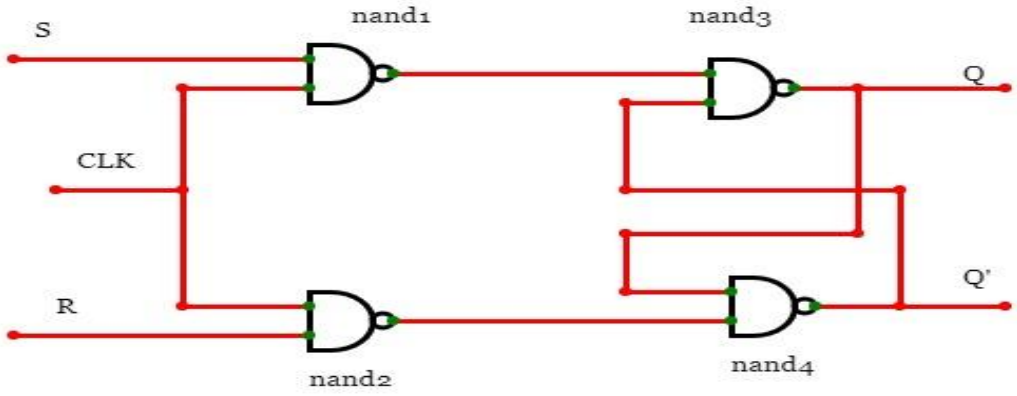
JK Flipflop

The J-K flip-flop is the most versatile of the basic flip flops. The JK flip flop is a gated SR flip-flop with the addition of a clock input circuitry that prevents the illegal or invalid output condition that can occur when both inputs S and R are equal to logic 1. Due to this additional clocked input, a JK flip-flop has four possible input combinations, “logic 1”, “logic 0”, “no change” and “toggle”.

TRUTH TABLE:

CLK	D	Q	Q’
0	0	0	1
1	0	0	1
0	1	0	1
1	1	1	0

JK FLIP FLOP:



TRUTH TABLE:

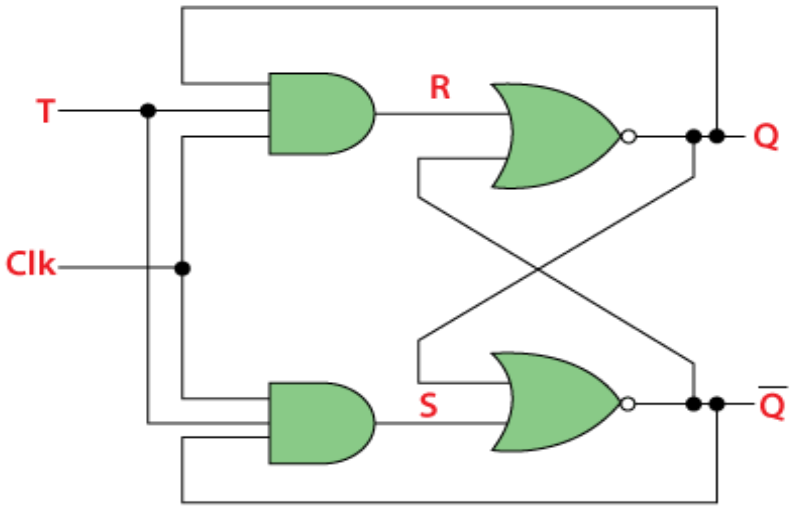
CLK	J	K	Qn+1
0	x	x	Qn
1	0	0	Qn
1	0	1	0
1	1	0	1
1	1	1	Qn’

T Flipflop

In T flip flop, "T" defines the term "Toggle". In SR Flip Flop, we provide only a single input called "Toggle" or "Trigger" input to avoid an intermediate state occurrence. Now, this flip-flop work as a Toggle switch. The next output state is changed with the complement of the present state output. This process is known as "Toggling"'.

We can construct the "T Flip Flop" by making changes in the "JK Flip Flop". The "T Flip Flop" has only one input, which is constructed by connecting the input of JK flip flop. This single input is called T. In simple words, we can construct the "T Flip Flop" by converting a "JK Flip Flop". Sometimes the "T Flip Flop" is referred to as single input "JK Flip Flop".

T FLIP FLOP:



TRUTH TABLE:

CLK	T	Qn+1
0	x	Qn
1	0	Qn
1	1	Qn’

PROGRAM:

SR FLIP-FLOP:

Source code:

```
module sr_ff (
    input clk,          // Clock input
    input rst,          // Reset (active high)
    input S, R,         // Set and Reset inputs
    output reg Q,       // Output
    output Qbar         // Complement of Q
);
    always @(posedge clk or posedge rst)
    begin
        if (rst)
        begin
            Q <= 1'b0; // Reset state
        end
        else
        begin
            case ({S, R})
                2'b00: Q <= Q;   // No change
                2'b01: Q <= 1'b0; // Reset
                2'b10: Q <= 1'b1; // Set
                2'b11: Q <= 1'bx; // Invalid condition
            endcase
        end
    end
   assign Qbar = ~Q; // Complement of Q
endmodule
```

OUTPUT:

Constraint code:

```
## Clock (100 MHz onboard oscillator)
set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports { clk }]
create_clock -period 10.00 -name sys_clk_pin -waveform {0 5} [get_ports { clk }]
## Reset button (BTNU instead of BTNC)
set_property -dict { PACKAGE_PIN T18 IOSTANDARD LVCMOS33 } [get_ports { rst }]
## Switches for S and R
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports { S }]
set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports { R }]
## LEDs for Q and Qbar
set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports { Q }]
set_property -dict { PACKAGE_PIN U19 IOSTANDARD LVCMOS33 } [get_ports { Qbar }]
```

D FLIP-FLOP:

Source code:

```
module d_ff (
    input clk,          // Clock input
    input rst,          // Reset (active high, BTNU)
    input D,            // Data input
    output reg Q,       // Normal output
    output Qbar         // Complement output
);
    assign Qbar = ~Q;
    always @(posedge clk or posedge rst)
    begin
        if (rst)
        begin
            Q <= 1'b0; // Reset Q to 0
        end
        else
        begin
            Q <= D; // On clock edge, Q takes D
        end
```

```
 end
endmodule
```

Constraint code:

```
## Clock (100 MHz onboard oscillator)
set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports { clk }]
create_clock -period 10.00 -name sys_clk_pin -waveform {0 5} [get_ports { clk }]
## Reset button (BTNU = T18)
set_property -dict { PACKAGE_PIN T18 IOSTANDARD LVCMOS33 } [get_ports { rst }]
## Input D from switch SW0
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports { D }]
## Outputs Q and Qbar to LEDs LD0, LD1
set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports { Q }]
set_property -dict { PACKAGE_PIN U19 IOSTANDARD LVCMOS33 } [get_ports { Qbar }]
```

JK FLIP-FLOP:

Source code:

```
module jk_ff (
    input clk,          // Clock input
    input rst,          // Reset (active high, BTNU)
    input J, K,         // JK inputs
    output reg Q,       // Normal output
    output Qbar         // Complement output
);
    assign Qbar = ~Q;

    always @(posedge clk or posedge rst)
    begin
        if (rst)
        begin
            Q <= 1'b0; // Reset output to 0
        end
        else
```

```
 begin
        case ({J, K})
           2'b00: Q <= Q;   // No change
           2'b01: Q <= 1'b0; // Reset
           2'b10: Q <= 1'b1; // Set
           2'b11: Q <= ~Q; // Toggle
        endcase
      end
   end
endmodule
```

Constraint code:

```
## Clock (100 MHz onboard oscillator)
set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports { clk }]
create_clock -period 10.00 -name sys_clk_pin -waveform {0 5} [get_ports { clk }]
## Reset button (BTNU = T18)
set_property -dict { PACKAGE_PIN T18 IOSTANDARD LVCMOS33 } [get_ports { rst }]
## Inputs J and K from switches SW0, SW1
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports { J }]
set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports { K }]
## Outputs Q and Qbar to LEDs LD0, LD1
set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports { Q }]
set_property -dict { PACKAGE_PIN U19 IOSTANDARD LVCMOS33 } [get_ports { Qbar }]
```

T FLIP-FLOP:

Source code:

```
module t_ff (input clk,         // Clock input
   input rst,         // Reset (active high, BTNU)
   input T,           // Toggle input
   output reg Q,      // Normal output
   output Qbar        // Complement output
);
```

```
 assign Qbar = ~Q;
always @(posedge clk or posedge rst)
begin
    if (rst)
    begin
        Q <= 1'b0; // Reset Q to 0
    end
    else
    begin
        if (T)
            Q <= ~Q; // Toggle Q when T=1
        else
            Q <= Q; // Hold state when T=0
    end
end
endmodule
```

Constraint code:

```
## Clock (100 MHz onboard oscillator)
set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports { clk }]
create_clock -period 10.00 -name sys_clk_pin -waveform {0 5} [get_ports { clk }]
## Reset button (BTNU = T18)
set_property -dict { PACKAGE_PIN T18 IOSTANDARD LVCMOS33 } [get_ports { rst }]
## Input T from switch SW0
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports { T }]
## Outputs Q and Qbar to LEDs LD0, LD1
set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports { Q }]
set_property -dict { PACKAGE_PIN U19 IOSTANDARD LVCMOS33 } [get_ports { Qbar }]
```

RESULT:

Thus the implementation of flip-flops was designed using verilog and simulated successfully.

EX NO: 9 EMBEDDED PROGRAMS FOR I/O INTERFACING USING PIC CONTROLLER

FLASHING LEDS WITH PIC

DATE:

AIM:

To write a program in Embedded C to flash LEDs using the PIC16F877A microcontroller kit.

APPARATUS REQUIRED:

PIC Microcontroller Trainer Kit, PC with MPLAB software, USB cable, Power Supply.

PROCEDURE:

1. Create a new project in MPLAB IDE for the PIC16F877A Microcontroller.
2. Create a new .c file to enter the code and add the file to the project.
3. Enter the required C program in the file and save it.
4. Build the project and check for errors.
5. Open the Flash Programmer Software and select the object file created in the project.
6. Program the flash of the microcontroller through the bootloader.
7. Press reset on the microcontroller kit and observe the result.

PROGRAM:

```
void main(void)
{
   long a;

   R_MAIN_UserInit();

   while(1U)
   {
      P6.3 = 1;
      P6.2 = 1;

      for(a = 0; a <= 0x3FFFF; a++);

      P6.3 = 0;
      P6.2 = 0;

      for(a = 0; a <= 0x3FFFF; a++);
   }
}
```

RESULT:

Thus, the program for flashing LEDs using the PIC16F877A microcontroller has been written in Embedded C and the output is verified.

EX NO: 10 A. Design a Stepper motor controller using PIC controller

DATE:

AIM:

To write a program in Embedded C to control a stepper motor using the PIC16F877A microcontroller kit.

APPARATUS REQUIRED:

PIC Microcontroller Trainer Kit, Stepper Motor, PC with MPLAB software, USB cable, Power Supply.

CONNECTIONS:

M1 - Stepper motor 1st wire or brown wire

M2 - Stepper motor 2nd wire or black wire

M3 - Stepper motor 3rd wire or yellow wire

M4 - Stepper motor 4th wire or orange wire

PROCEDURE:

1. Connect the Motor Control Interface Card to the PIC Microcontroller Kit.
2. Connect the stepper motor to the output of the motor driver IC in the Motor Control card
3. Create a new project in MPLAB IDE for the PIC16F877A Microcontroller.
4. Create a new .c file to enter the code and add the file to the project.
5. Enter the required C program in the file and save it.
6. Build the project and check for errors.
7. Open the Flash Programmer Software and select the object file created in the project.
8. Program the flash of the microcontroller through the bootloader.
9. Press reset on the microcontroller kit and observe the result.

PROGRAM:

```
#include<htc.h>
#define Stepper PORTD
void main(void)
{       longi =0,Count = 0;
        TRISD = 0x00;          // Port Direction Register – Set as Output port
                while(1)
        {
                Stepper = 0x16;
                for(i=0;i<=0x8000;i++);                 // Small Delay
                Stepper = 0x1C;
                for(i=0;i<=0x8000;i++);                 // Small Delay
                Stepper = 0x19;
                for(i=0;i<=0x8000;i++);                 // Small Delay
Stepper = 0x13;
                for(i=0;i<=0x8000;i++);                 // Small Delay
        }
}
```

RESULT:

Thus, the program for Stepper motor control using the PIC16F877A microcontroller has been written in Embedded C and the output is verified.

EX NO: 10 B. Design a DC motor controller using PIC controller

DATE:

AIM:

To write a program in Embedded C for PWM based Speed control of a DC motor using the PIC16F877A microcontroller kit.

APPARATUS REQUIRED:

PIC Microcontroller Trainer Kit, DC Motor, PC with MPLAB software, USB cable, Power Supply.

CONNECTIONS:

1. Connect the DC Motor in PWM Green Connector.
2. Connect the one terminal in M6 and another terminal in CM1

PROCEDURE:

1. Connect the Motor Control Interface Card to the PIC Microcontroller Kit.
2. Connect the DC motor to the output of the motor driver IC in the Motor Control card
3. Create a new project in MPLAB IDE for the PIC16F877A Microcontroller.
4. Create a new .c file to enter the code and add the file to the project.
5. Enter the required C program in the file and save it.
6. Build the project and check for errors.
7. Open the Flash Programmer Software and select the object file created in the project.
8. Program the flash of the microcontroller through the bootloader.
9. Press reset on the microcontroller kit and observe the result.

PROGRAM:

```
#include<htc.h>
#define MOTOR RD4
#define Duty_Cycle    80        // Put values between 0 and 100
void main(void)
{
        longi =0;
        TRISD = 0;     // Port Direction Register – Set as Output port
                while(1)
        {
                MOTOR = 0;
                for(i=0;i<Duty_Cycle;i++);                      // On Time

                MOTOR = 0x1C;
                for(i=0;i<(100 - Duty_Cycle);i++);              // Off Time
        }
}
```

RESULT:

Thus, the program for PWM based Speed control of a DC motor using the PIC16F877A microcontroller has been written in Embedded C and the output is verify

CONFIGURATION DETAILS:

1. Clock Settings
   - Clock Type – High Speed System Clock
   - Disable Internal OCO (On – Chip Oscillator)
   - Enable External Crystal oscillator of 20 Mhz
2. Watch Dog Timer – Disable
3. Timer Settings
   - Channel 0 – Multiple PWM Output (Master)
   - Channel 2 to 7 – Multiple PWM Output (Slave)
     - Channel 0
       i. PWM Cycle - 1ms
       ii. Generate Interrupt – Low Priority
     - Channel 2
       i. Duty Cycle – 99%
       ii. Output Settings - Output Value = 0 ,Output Level = High
       iii. Generate Interrupt – Low Priority
     - Channel 3
       i. Duty Cycle – 80%
       ii. Output Settings - Output Value = 0 ,Output Level = High
       iii. Generate Interrupt – Low Priority
     - Channel 4
       i. Duty Cycle – 60%
       ii. Output Settings - Output Value = 0 ,Output Level = High
       iii. Generate Interrupt – Low Priority
     - Channel 5
       i. Duty Cycle – 50%
       ii. Output Settings - Output Value = 0 ,Output Level = High
       iii. Generate Interrupt – Low Priority
     - Channel 6
       i. Duty Cycle – 25%
       ii. Output Settings - Output Value = 0 ,Output Level = High
       iii. Generate Interrupt – Low Priority
     - Channel 7
       i. Duty Cycle – 10%
       ii. Output Settings - Output Value = 0 ,Output Level = High
       iii. Generate Interrupt – Low Priority

CONNECTIONS:

1. LEDs
   - P1.5 - LED 2 - Timer Channel2 (T002)
   - P1.4 - LED 3 - Timer Channel3 (T003)
   - P1.3 - LED 4 - Timer Channel4 (T004)
   - P1.2 - LED 5 - Timer Channel5 (T005)
   - P1.1 - LED 6 - Timer Channel6 (T006)
   - P1.0 - LED 7 - Timer Channel7 (T007)

EX NO: 11. Generate 3-phase PWM signals and demonstrate the utility of PWM with high bright LED lights using RL78

DATE:

AIM:

To write a program in Embedded C to generate multiple synchronized PWMs with the RL78 microcontroller kit.

APPARATUS REQUIRED:

RL78 Embedded Application Development System, PC with CubeSuite software, EZ-Recon Emulator, USB Cable, Power Supply.

PROCEDURE:

1. Connect the required devices to the various pins of the RL78 microcontroller as per the connection.
2. Create a new project in CubeSuite IDE for the RL78/G13 – R5F100GEA Microcontroller.
3. Generate initialization code using the inbuilt code generator for various on-chip peripherals and I/O devices.
4. Enter the required C program in the main file and save it.
5. Build the project and check for errors.
6. Download the program using the emulator.
7. Execute the program and verify the output.

PROGRAM:

```
void R_TAU0_Channel0_Start(void);
void main(void)
{
        R_TAU0_Channel0_Start();  //Start Timer Channel 0
        while(1);
}
```

RESULT:

Thus, the program to generate multiple synchronized PWMs using the RL78 microcontroller has been written in Embedded C and the output is verified.

CONFIGURATION DETAILS:

1. Clock Settings
   - Clock Type – High Speed System Clock
   - Disable Internal OCO(On – Chip Oscillator)
   - Enable External Crystal oscillator of 20 Mhz
2. Watch Dog Timer – Disable
3. Port Settings
   - Port 1.0 to 1.7 - Output
   - Port 14.0,14.6,14.7 - Output
4. ADC Settings
   - ADC Resolution - 10 Bit mode
   - VREF(+) - Internal Reference
   - VREF(-) - VSS
   - Software Trigger Mode
   - One Shot - Select Mode
   - Analog Input Selection - ANI0 to ANI2
   - A/D Channel Selection - ANI2

CONNECTIONS:

1. LCD Interface – Data Bus Lines
   - D0 - Port 1.0
   - D1 - Port 1.1
   - D2 - Port 1.2
   - D3 - Port 1.3
   - D4 - Port 1.4
   - D5 - Port 1.5
   - D6 - Port 1.6
   - D7 - Port 1.7
2. LCD Interface – Data Bus Lines
   - RS - Port 14.0
   - RW - Port 14.6
   - E - Port 14.7
3. Temperature Sensor LM35 – Port 2.2

EX NO: 12 **Measure room temperature and display the same in a LCD with keyboard interaction using RL 78**

DATE:

AIM:

To write a program in Embedded C to measure room temperature using the temperature sensor LM35 and to display it in an LCD using the RL78 microcontroller kit.

APPARATUS REQUIRED:

RL78 Embedded Application Development System, PC with CubeSuite software, EZ-Recon Emulator, USB Cable, Power Supply.

PROCEDURE:

1. Connect the required devices to the various pins of the RL78 microcontroller as per the connection.
2. Create a new project in CubeSuite IDE for the RL78/G13 – R5F100GEA Microcontroller.
3. Generate initialization code using the inbuilt code generator for various on-chip peripherals and I/O devices.
4. Add library files for LCD initialization and control to the project.
5. Enter the required C program in the main file and save it.
6. Build the project and check for errors.
7. Download the program using the emulator.
8. Execute the program and verify the output.

PROGRAM:

```
#include "LCD.h"

void main(void)
{
   char t[16] = " . ";
   unsigned long int a;
   unsigned long int b;

   InitializeLCD();
   CursorOFF();
   DisplayLCD(1, "Room Temperature");
   R_ADC_Set_OperationOn();

   while(1U)
   {
      ADCS = 1U;
      while(ADCS == 0);

      a = (ADCR >> 6U);
      a = (a * 1400) / 1023;

      t[6] = a / 100 + '0';
      a -= (a / 100) * 100;

      t[7] = a / 10 + '0';
      t[9] = a % 10 + '0';

      DisplayLCD(2, t);

      for(b = 0; b < 0x1ffff; b++);
   }
}
```

RESULT:

Thus, the program for room temperature measurement and display using the RL78 microcontroller has been written in Embedded C and the output is verified.

CONFIGURATION DETAILS:

1. **Clock Settings**
   - Clock Type – High Speed System Clock
   - Disable Internal OCO(On – Chip Oscillator)
   - Enable External Crystal oscillator of 20 Mhz
2. **Watch Dog Timer** – Disable
3. **Port Settings**
   - Port 1.0 to 1.7 - Output
   - Port 14.0,14.6,14.7 - Output
4. **Timer** – Channel 7 - Input Pulse Interval Measurement
   - TI07 Interval range - 3.2uS to 209.714mS
   - TI07 Input Edge Setting - Falling Edge
   - Interrupt – Enable end of capture Interrupt, Set Low Priority

CONNECTIONS:

1. LCD Interface – Data Bus Lines
   - D0 - Port 1.0
   - D1 - Port 1.1
   - D2 - Port 1.2
   - D3 - Port 1.3
   - D4 - Port 1.4
   - D5 - Port 1.5
   - D6 - Port 1.6
   - D7 - Port 1.7
2. LCD Interface – Data Bus Lines
   - RS - Port 14.0
   - RW - Port 14.6
   - E - Port 14.7
3. Timer Input – Port 4.1 (TI07)

EX NO: 13. Design an Embedded system to measure the unknown signal frequency using Timer/Counter of RL78.

DATE:

AIM:

To write a program in Embedded C to measure the frequency of a square wave input and to display it in an LCD using the RL78 microcontroller kit.

APPARATUS REQUIRED:

RL78 Embedded Application Development System, PC with CubeSuite software, EZ-Recon Emulator, USB Cable, Power Supply.

PROCEDURE:

1. Connect the required devices to the various pins of the RL78 microcontroller as per the connection.
2. Create a new project in CubeSuite IDE for the RL78/G13 – R5F100GEA Microcontroller.
3. Generate initialization code using the inbuilt code generator for various on-chip peripherals and I/O devices.
4. Add library files for LCD initialization and control to the project.
5. Enter the required C program in the main file and save it.
6. Build the project and check for errors.
7. Download the program using the emulator.
8. Execute the program and verify the output.

PROGRAM:

```
#include "LCD.h"

unsigned long int Frequency;
extern volatile uint32_t g_tau0_ch7_width;

void main(void)
```

```
{
   char t[25] = "Frequency: ";
   unsigned long int a;

   InitializeLCD();
   DisplayLCD(1, "Freq. Counter: ");
   EI();

   while(1)
   {
      Frequency = 635000 / g_tau0_ch7_width;

      t[10] = Frequency / 100000 + '0';
      Frequency -= (Frequency / 10000) + 100000;

      t[11] = Frequency / 10000 + '0';
      Frequency -= (Frequency / 10000) + 10000;

      t[12] = Frequency / 1000 + '0';
      Frequency -= (Frequency / 10000) + 1000;

      t[13] = Frequency / 100 + '0';
      Frequency -= (Frequency / 10000) + 100;

      t[14] = Frequency / 10 + '0';
      t[15] = Frequency / 10 + '0';

      DisplayLCD(2, t); // Display the frequency
      for(a = 0; a < 0x5ffff; a++); // A small delay
   }
}
```

RESULT:

Thus, the program for frequency measurement and display using the RL78 microcontroller has been written in Embedded C and the output is verified.

CONFIGURATION DETAILS:

1. Clock Settings
    - Clock Type – High Speed System Clock
    - Disable Internal OCO(On – Chip Oscillator)
    - Enable External Crystal oscillator of 20 Mhz
2. Watch Dog Timer – Enable
    - Overflow Time – 4369mS
    - Window Open Period – 1000%
    - Enable interrupt at 75% overflow – Low Priority
3. Port Settings
    - Port 1.0 to 1.7 - Output
    - Port 6.0,6.1 - Input

CONNECTIONS:

1. LEDs
    - LED2 - Port 1.0
    - LED3 - Port 1.1
    - LED4 - Port 1.2
    - LED5 - Port 1.3
    - LED6 - Port 1.4
    - LED7 - Port 1.5
    - LED8 - Port 1.6
    - LED9 - Port 1.7
2. Push Button Switch
    - Switch18 - Port 6.0
    - Switch19 - Port 6.1

EX NO: 14 Demonstrate the usage of Watchdog timers and voltage detection facilities of RL78 in an application.

DATE:

AIM:

To write a program in Embedded C to demonstrate the application of watch dog timer using the RL78 microcontroller kit.

APPARATUS REQUIRED:

RL78 Embedded Application Development System, PC with CubeSuite software, EZ-Recon Emulator, USB Cable, Power Supply.

PROCEDURE:

1. Connect the required devices to the various pins of the RL78 microcontroller as per the connection.
2. Create a new project in CubeSuite IDE for the RL78/G13 – R5F100GEA Microcontroller.
3. Generate initialization code using the inbuilt code generator for various on-chip peripherals and I/O devices.
4. Enter the required C program in the main file and save it.
5. Build the project and check for errors.
6. Download the program using the emulator.
7. Execute the program and verify the output.

PROGRAM:

```
#define Firstswitch P6.0
#define Secondswitch P6.1
#define FirstLED P1.0
#define SecondLED P1.7

unsigned char b[20] = {
   0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80,
   0x40, 0x20, 0x10, 0x08, 0x04, 0x02, 0x01
};

void R_WDT_Create(void);
void R_WDT_Restart(void);
void OutputPort(unsigned char a);

void main(void)
{
   unsigned char a, c;
   long d; // loop enable
   a = 0;
   c = 1;

   while (c)
   {
      P1 = b[a]; // Sequentially switch on/off LEDs from 1 to 8
      a++;

      if (a > 13)
      {
         a = 0;
      }

      if (Firstswitch == 0) // Check first switch status
      {
         c = 0; // Clear the loop enable
         while (Firstswitch == 0); // Wait for switch release
      }
```

```
R_WDT_Restart; // Restart watchdog timer
      for (d = 0; d < 0x1FFFF; d++); // Delay
   }

   P1 = 0; // Clear all LEDs
   c = 0;

   R_WDT_Create(); // Enable watchdog timer

   while (1)
   {
      FirstLED ^= 1; // Toggle 1st LED
      for (d = 0; d < 0xFFFF; d++); // Small delay

      if (c == 0)
      {
         R_WDT_Restart(); // Restart watchdog timer
      }

      if (Secondswitch == 0) // Check switch SW3 status
      {
         EightLED = 1; // If pressed, switch on 8th LED
         c = 1; // Stop refreshing watchdog timer
      }
   }
}
```

RESULT:

Thus, the program to demonstrate the application of watch dog timer using the RL78 microcontroller has been written in Embedded C and the output is verified.

EX NO: 15 Interface ADC with embedded system trainer kit.

DATE:

AIM:

To write a program in Embedded C to Convert analog signal to digital signal using the ARM7 microcontroller kit.

APPARATUS REQUIRED:

ARM7 Microcontroller Trainer Kit, PC with KEILμVISION software, USB cable, Power Supply.

PROCEDURE:

1. Create a new project in KEIL IDE for the ARM7 Microcontroller.
2. Create a new .c file to enter the code and add the file to the project.
3. Enter the required C program in the file and save it.
4. Build the project and check for errors.
5. Open the Flash Magic Software and select the hex file created in the project.
6. Program the flash of the microcontroller through the bootloader and observe the result

PROGRAM:

```
#include <lpc214x.h>
#include "lcd.h"
#include "adclcd.h"
#include "adcseg.h"

#define led_DIR   IO0DIR
#define led_SET   IO0SET
#define led_CLR   IO0CLR
#define swt_DIR   IO1DIR
#define swt_SET   IO1SET
#define swt_CLR   IO1CLR
#define l8        (1<<30)
#define l9        (1<<31)
#define led_pin   0xC0000000
#define swt_pin   0x01800000
```

```
unsigned int a = 0, b = 0;
int getpinstate1(int pinnumber);

int main(void)
{
   led_DIR |= led_pin;
   swt_DIR &= ~(swt_pin);
   lcdinit();

   while(1)
   {
      if(!getpinstate1(sw3) && !getpinstate1(sw4))
      {
         led_CLR = l8 | l9;
         a = 3;

         if(a != b)
         {
            lcdinit();
            b = a;
         }

         lcddisplay(0x82, "ENIXS TECHNOLOGY", 16);
         lcddisplay(0xC3, "ADC-0809 DEMO", 13);
         lcddisplay(0x94, "SW3-ADC Result:LCD", 18);
         lcddisplay(0xD4, "SW4-ADC Result:SEG", 18);
      }

      if(getpinstate1(sw3) && !getpinstate1(sw4))
      {
         led_CLR = led_pin;
         led_SET = l8;
         led_CLR = l9;
         a = 1;

         if(a != b)
         {
            lcdinit();
            b = a;
         }
```

```
adclcd();
        }

        if(!getpinstate1(sw3) && getpinstate1(sw4))
        {
           led_CLR = led_pin;
           led_SET = l9;
           led_CLR = l8;
           a = 2;

           if(a != b)
           {
              lcdinit();
              b = a;

              lcddisplay(0x80, "Multichannel ADC CH0", 20);
              lcddisplay(0xC2, "Toggle Swt to", 13);
              lcddisplay(0x98, "7SEG Mode", 9);
           }

           adcseg();
        }
    }
}

int getpinstate1(int pinnumber)
{
    int pinblockstate = IO1PIN;
    int pinstate = (pinblockstate & (pinnumber));

    return pinstate;
}
```

RESULT:

Thus, the program for analog signal to digital signal using the ARM7 microcontroller kit, has been written in Embedded C and the output is verified.

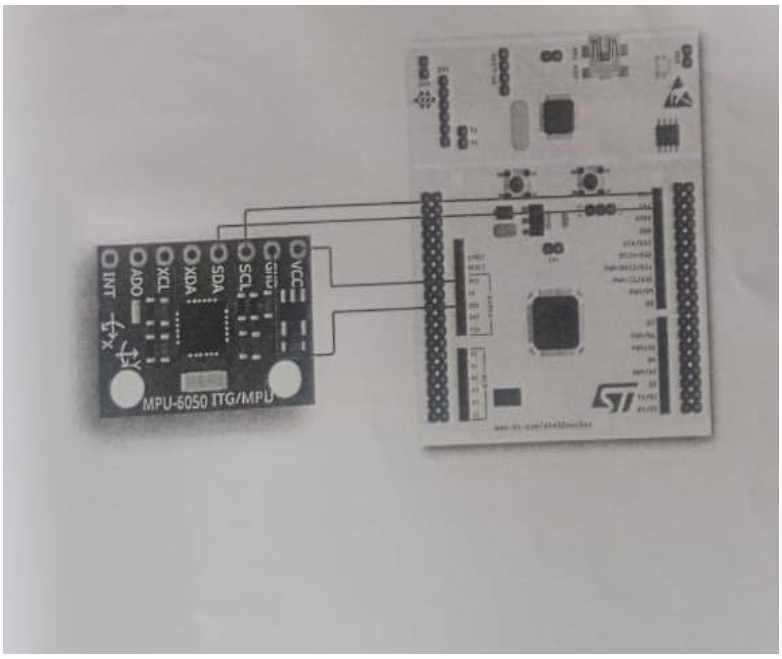
EX NO: 16 **Interface 3-axis motion & vibration sensor with STM32 Nucleo Board**

DATE:

AIM:

The goal of this experiment is to train a debugger so as to build our machine learning model on the NanoEdge AI Studio for detecting the abnormal vibration pattern in machines.

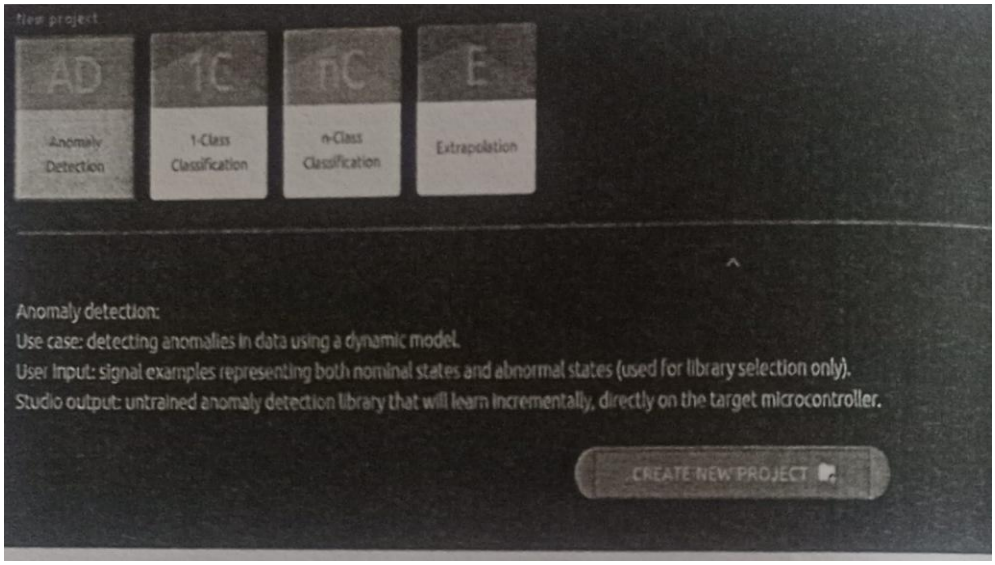
CONNECTION DIAGRAM:



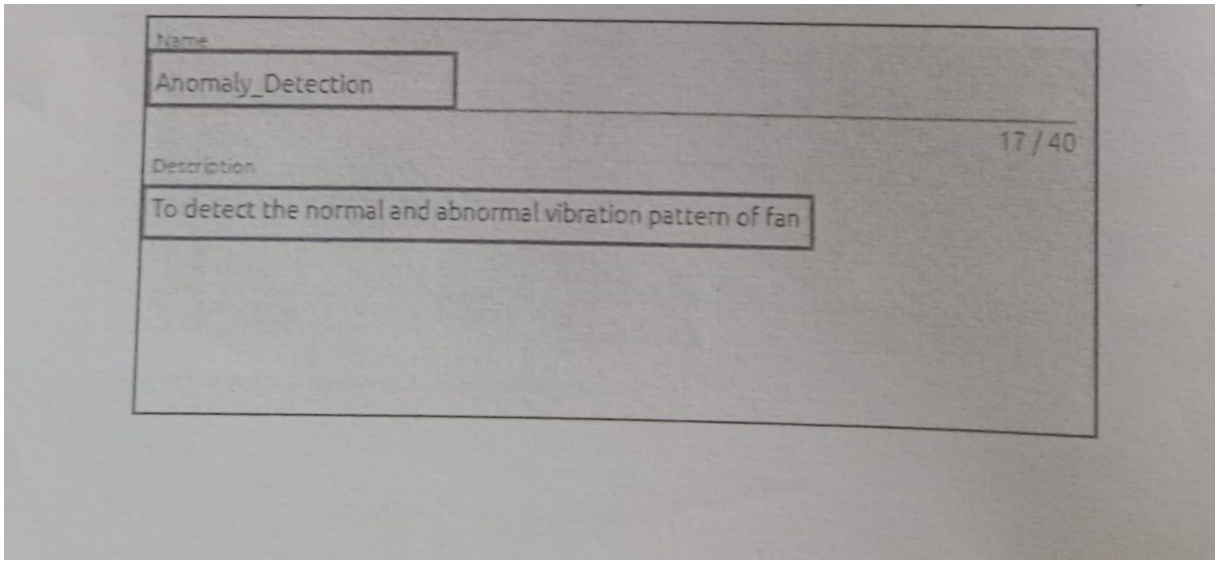
PROCEDURE:

Procedure (NanoEdge AI Studio):

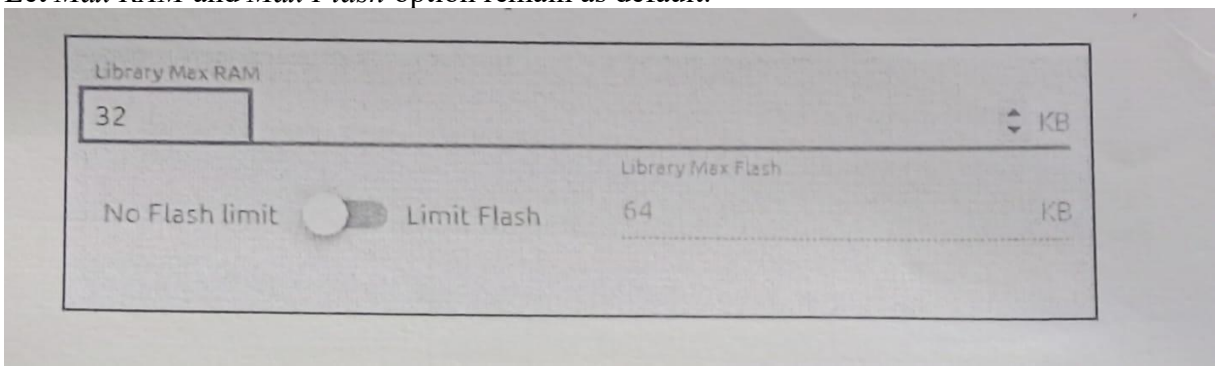
1. Open NanoEdge AI Studio.
2. Select *Anomaly Detection* project type and select *Create New Project*.



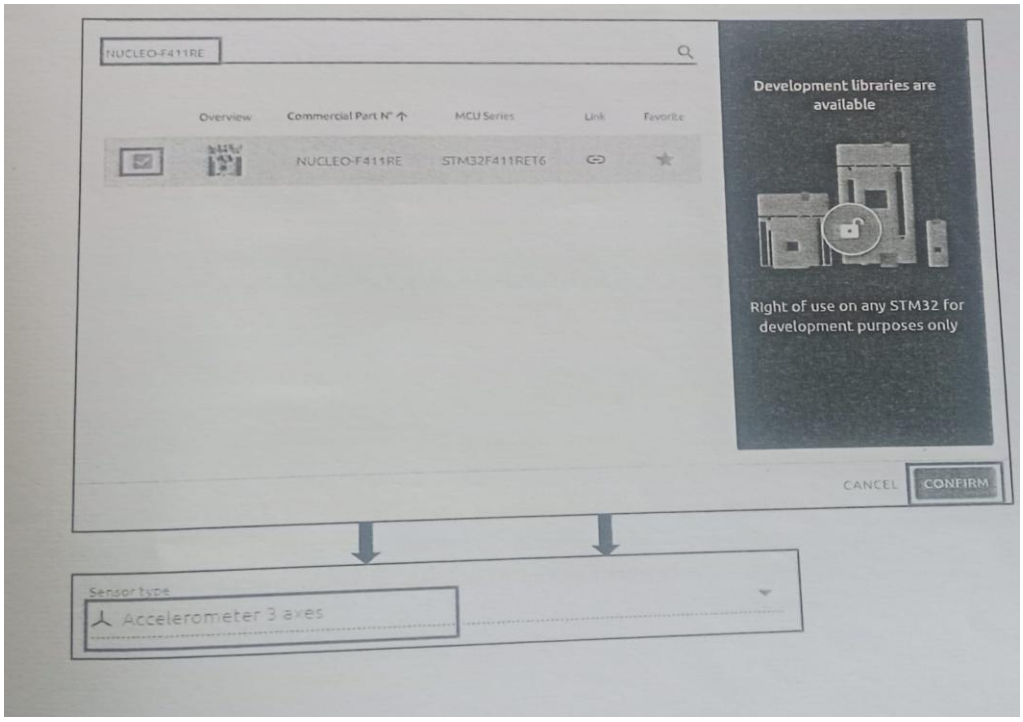
3. A new window will open. On the first step, **Project Settings**, name the project as *Anomaly Detection*.



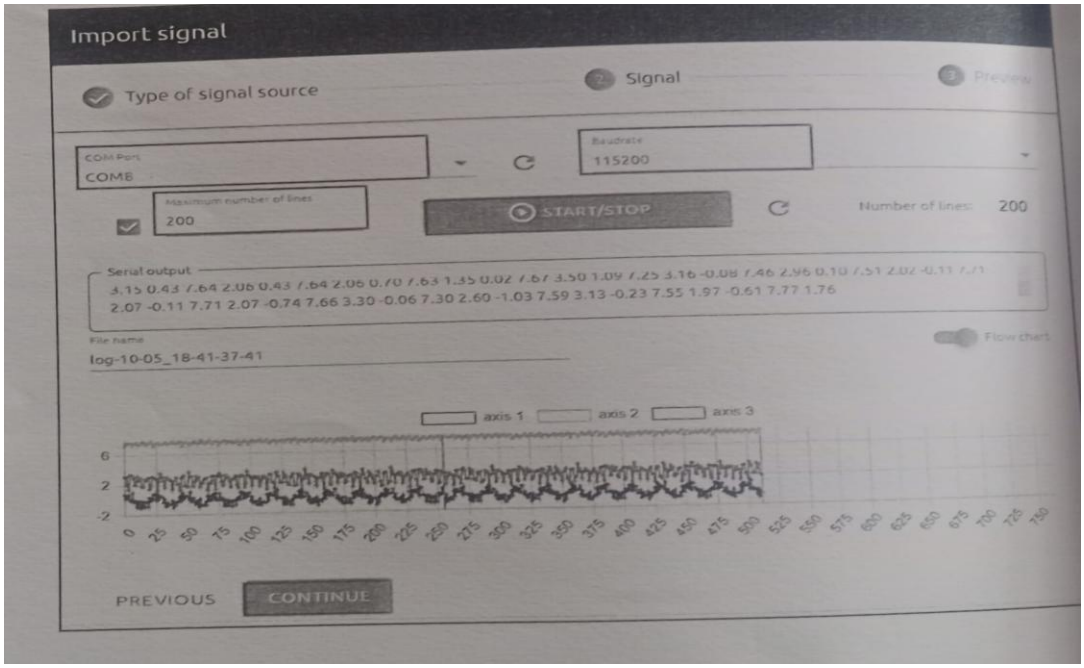
4. Let *Max RAM* and *Max Flash* option remain as default.



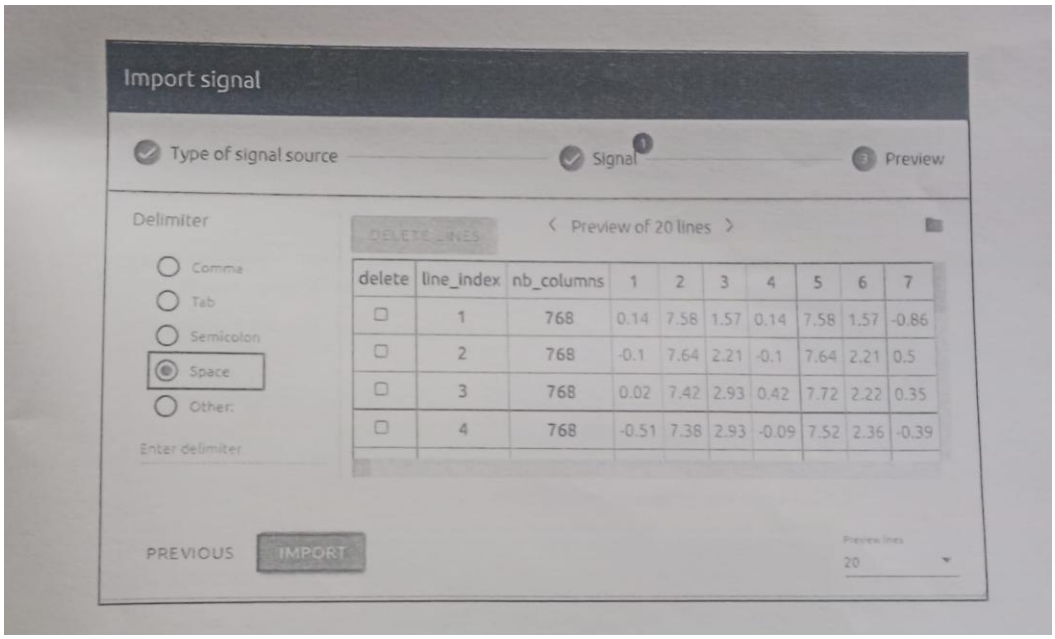
5. In *Target* section, select your proper **STM32 Nucleo board**. In *Sensor Type* section, select the type of sensor you are working with. For this experiment, you'll have to choose **Accelerometer 3-axis**.



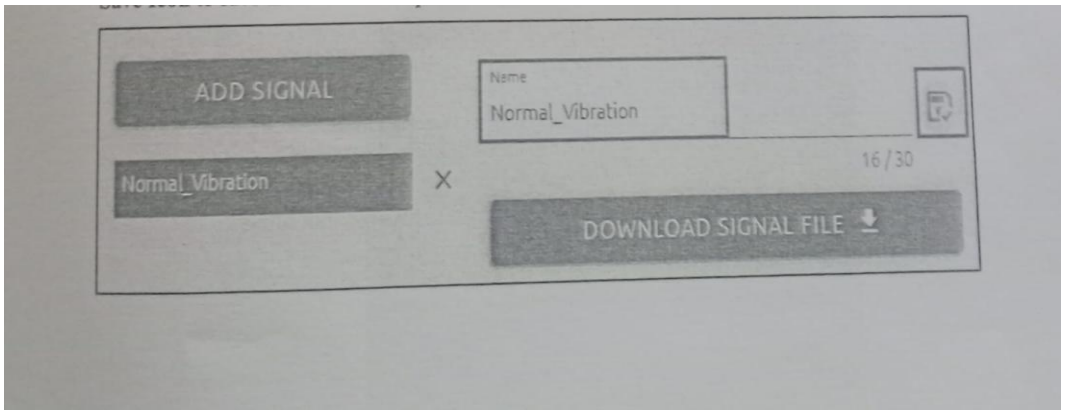
6. Now you can add your *board* (e.g., 115200). Select the correct **COM Port** based on the one your board uses.Manage it as **115200** in *Baud Rate*. In *Maximum number of lines* enter the value of 200.This setting allows the system2 to collect 200 lines of data before overwriting the vibration samples.Click on **Continue** to move onto acquiring the vibration samples.



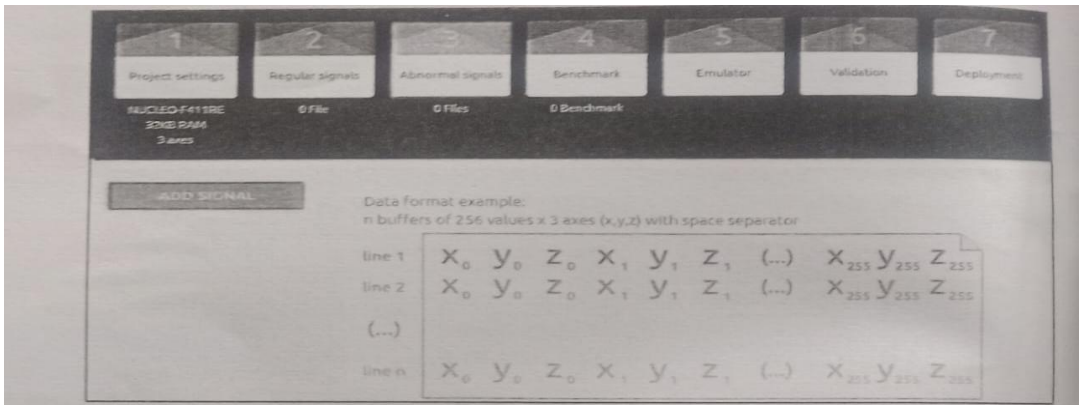
7. In the window, select the *Delimiter* as **Space**. If any line has some corrupted values, click *Delete it*.Select **Preview Lines** as 200 to check all the data you have collected, and click **Import**.



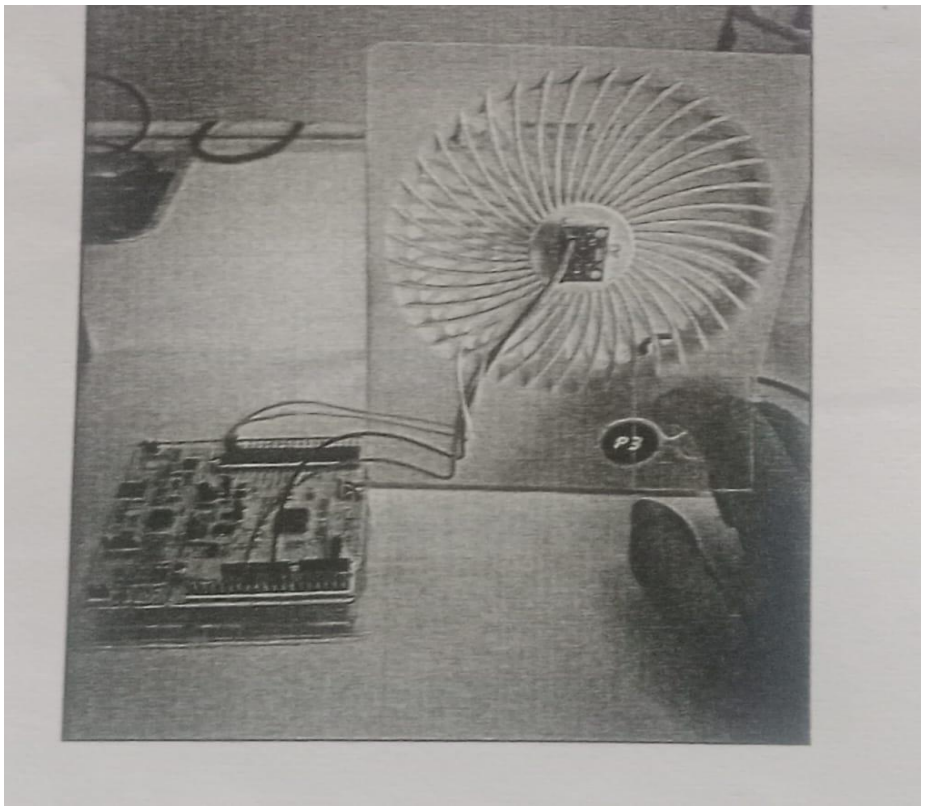
8. In the next window, rename the sample under *Name* as **Normal_Vibration** and click on the **Blue ADD SIGNAL** button.



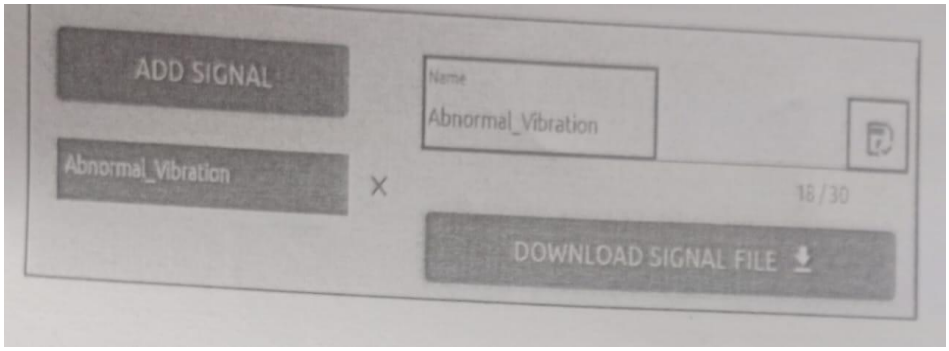
9. Now select *Abnormal Signal* and click on *Add Signal again.*



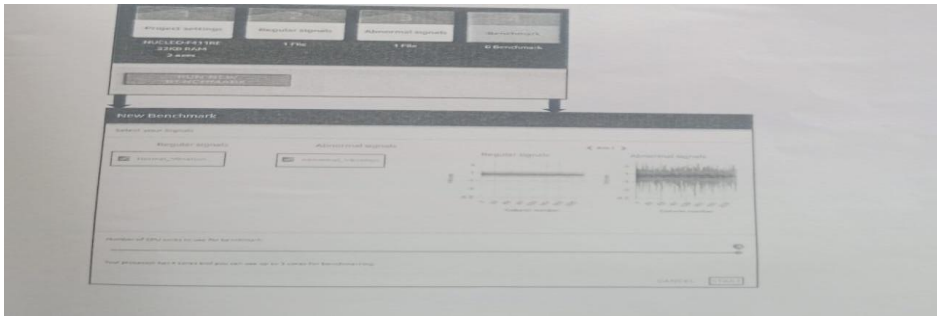
10. Turn on the fan. As it starts, demonstrate by your finger and start collecting data for the abnormal vibration. Collect signals from sensors in the axes *X*, *Y*, and *Z*.



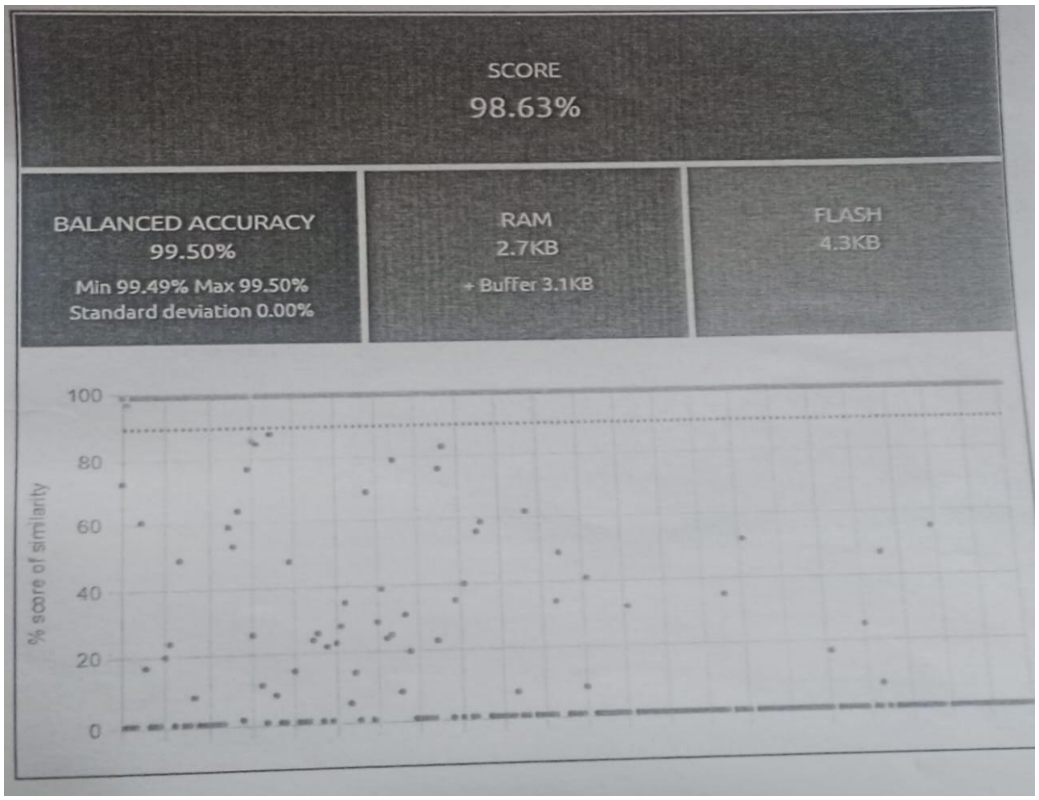
11. Now turn off the fan and name the file as *Abnormal Vibration.*



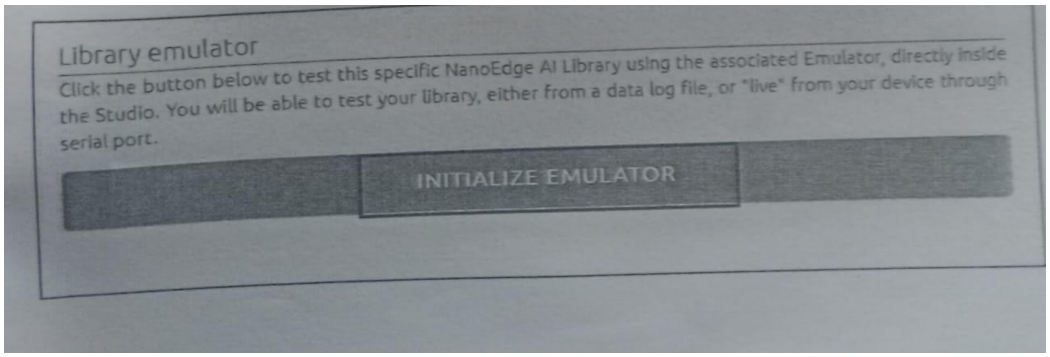
12. In the fourth step of the *NeuralEdge AI Studio* Benchmark, click on *Run Benchmark*. Next select *Regular Signals* and *Abnormal Signals* and click on *Start.*



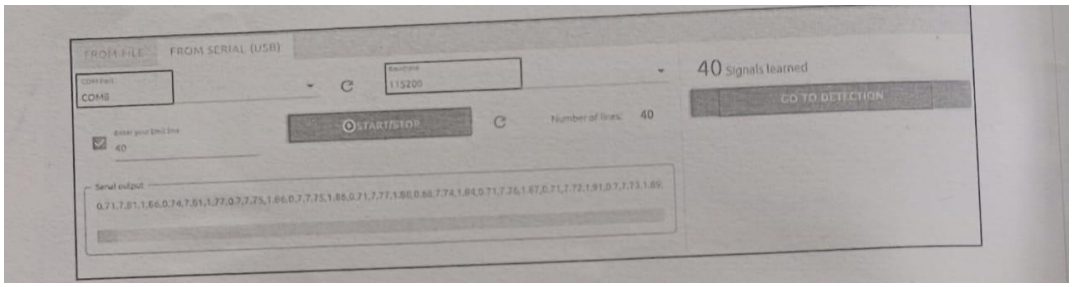
13.After the Benchmark has reached a Score of above 95%, stop the benchmark process



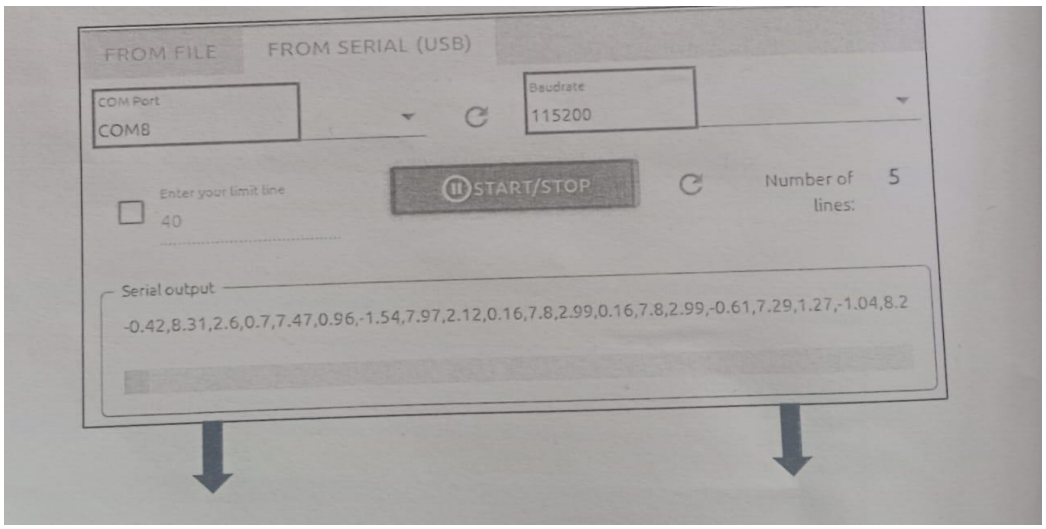
14. After the benchmark process is done, go to the fifth page: *Emulator*.
Click on *Initialize Emulator* to move on to test the library you have selected on the NeuralEdge AI Studio.



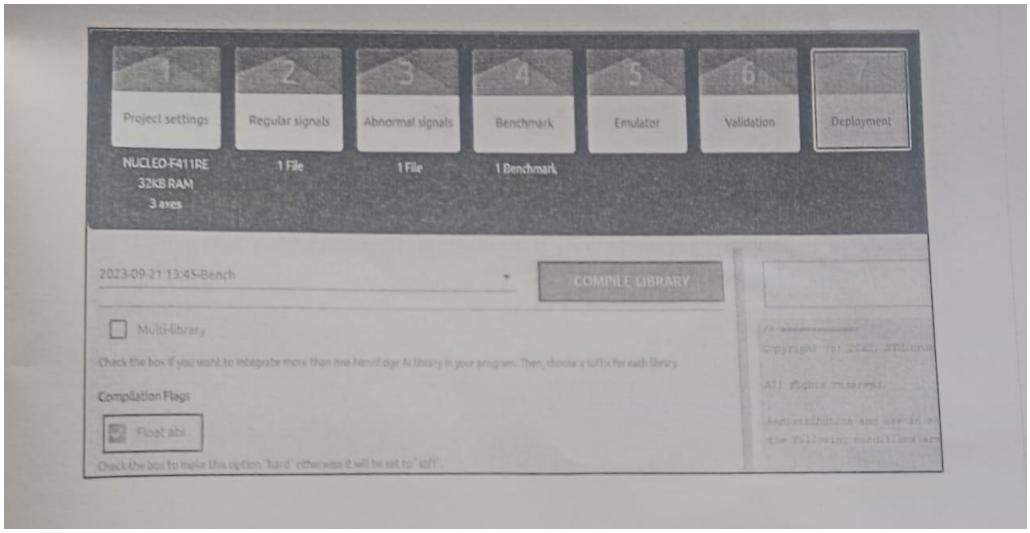
15. In the new window, select *From Serial (USB)*.Select *COM Port* according to the info from *Device Manager.*Select *Baud rate = 115200.*Click *START/STOP* to start reading data from the device.Number of lines: Set to **40**.Short the fan and then collect vibration information for 40 cycles.Now select *GO TO DETECTION.*



16. In the *Detection panel*, select *From Serial (USB).*Select *COM Port* according to info from *Device Manager.*Select *Baud rate = 115200.*Click *START/STOP* to begin detection.When 100% similarity is reached, it shows detected vibration as *Normal* or *Abnormal.*This is displayed in the *Serial Output* window as data values.



17. Finally go to Diplomasim stage, select Flochart, and click on Compile Library. Check or Get Library and save your Library on the PC



RESULT:

Thus, the program to interface the 3-axis motion and vibration sensor with the STM32 Nucleo board has been written in Embedded C, and the output results were successfully verified.